SWIFT SHIP TRACKER

Salesforce Developer Edition · Prompt Builder · Flow Builder · Agentforce

Abstract

Parceltracking information in many logistics settings is scattered across logbooks, spreadsheets, telephone queues, and standalone software. This fragmentation slows support teams, increases data-entry errors, and erodes customer trust. This report documents **SwiftShip Tracker**, an AI-enabled parcel management prototype built on the Salesforce Developer Edition platform as an academic project in the Department of Computer Science and Engineering.

The system consolidates parcel, delivery, sender, and receiver information into four custom objects, exposes them through a dedicated Lightning application with custom tabs, and uses Salesforce Prompt Builder to turn live record fields into a grounded natural-language tracking update. An auto-launched Flow (Parcel Details – V1) looks up a parcel by its auto-numbered Parcel ID, passes the record to the prompt template, and returns the generated text through an output variable. Agentforce is configured with a dedicated Parcel Tracker subagent so that users can request tracking information conversationally. Access is governed through a custom permission set, field-level security, and a role hierarchy.

Testing in the Developer Edition environment confirmed record creation, record retrieval by Parcel ID, merge-tag resolution, and flow output assignment for the test record P-001 (Laptop, status Booked, weight 2 kg, estimated delivery 10/17/2026). The report also records the observations made during testing, including a preview-stage finding that the agent must be bound to the custom parcel action before it can answer from Parcel__c data, and sets out a roadmap covering telematics, omnichannel notifications, a customer portal, and multi-carrier integration.

Keywords: Salesforce, CRM, Agentforce, Prompt Builder, Flow Builder, Apex, parcel tracking, grounded generative AI, logistics.

1. Project Background and Operational Context

Inmodern logistics and enterprisesupply chainmanagement, centralising fragmented parcel tracking mechanisms into a unified Customer Relationship Management (CRM) architecture is critical for operational efficiency, data integrity, and customer retention. Logistics enterprises frequently operate across disparate operational silos, where booking details, transit milestones, sender/receiver profiles, and delivery status updates reside in disconnected software applications or unindexed spreadsheets. Unifying these mechanisms within an enterprise CRM platform establishes a definitive single source of truth. This centralised architecture provides real-time operational visibility across the complete parcel lifecycle, eliminates manual administrative overhead, accelerates support desk response times, and builds a foundation for scalable workflow automation and generative artificial intelligence.

The traditional operational baseline in legacy logistics environments is severely constrained by information fragmentation. When parcel tracking data is distributed across manual logbooks, isolated spreadsheets, telephone support queues, and proprietary localised software, severe operational friction ensues. Field delivery personnel lack real-time mechanisms to log status transitions, while customer support teams must execute manual lookups across disconnected databases merely to address basic status inquiries. This results in high administrative costs, elevated data-entry error rates, delayed issue resolution, and degraded customer trust.

To overcome these challenges, the SwiftShip Tracker platform was architected as an AI-enabled parcel management prototype on the Salesforce platform. The operational parameters and technical details of the implementation are presented below:

Technical Parameter	System Specification & Implementation Details
Platform Environment	Salesforce Developer Edition
Core Custom Modules	Parcel (Parcel__c), Delivery (Delivery__c), Sender (Sender__c), Receiver (Receiver__c)
Automation Engine	Salesforce Flow Builder (Auto-Launched Flows)
Artificial Intelligence Layer	Agentforce + Salesforce Prompt Builder
User Interface Architecture	Salesforce Lightning Application & Dedicated Lightning Record Pages
Security Architecture	Profiles, Permission Sets, Role Hierarchy, Field-Level Security (FLS)
Project Type & Context	Academic / Prototype Implementation
Academic Department	Department of Computer Science and Engineering
Implementation Timeline	Academic Year 2026–2027

The primary strategic objective of the SwiftShip Tracker system is to resolve tracking fragmentation by integrating parcel booking, status management, stakeholder recording, and conversational customer support into a unified Salesforce architecture.

Strategic Objectives

- Centralise parcel, delivery, sender, and receiver records within a relational CRM data schema.
- Streamline end-to-end parcel lifecycle tracking from booking to final delivery confirmation.
- Provide real-time shipment status and estimated delivery metrics to internal staff and customer service channels.
- Automate operational data retrieval and payload routing through low-code process flows.
- Enable natural language conversational tracking using grounded generative AI models.
- Establish data foundations to drive executive visibility through native Salesforce reports and dashboards.
- Enforce enterprise-grade security controls and field-level access boundaries across all user personas.

Implementation Scope

- Datamodellingandcustom object configuration for Parcel__c, Delivery__c, Sender__c, and Receiver__c.
- Design, grounding, and activation of AI prompt templates in Prompt Builder.
- Configuration of auto-launched orchestration flows in Flow Builder to connect CRM data records to AI actions.
- Implementation of governance controls, including custom permission sets for Flow access execution.
- Development of custom Lightning tabs and record page layouts for streamlined user navigation.
- Empirical validation and test execution within a controlled Salesforce Developer Edition environment.

1.1 Problem Statement

Parcellogistics depends on answering one question reliably: where is my parcel and when will it arrive? In fragmented environments that question can only be answered by a person who knows which spreadsheet, logbook, or system holds the latest information. The consequences are measurable in daily operations:

- **Duplicate and inconsistent records:** The same parcel may exist in several places with different statuses or delivery estimates.
- **Slow support response:** Support staff spend time searching instead of resolving, so simple status inquiries consume disproportionate effort.
- **No self-service:** Customers cannot check progress without contacting the organisation, which increases call volume.
- **Weak auditability:** Without field history and structured status values, it is difficult to show when and by whom a status changed.
- **Barriers to automation:** Unstructured data cannot easily feed reports, notifications, or AI assistants.

SwiftShip Tracker addresses these problems by storing parcel data in a structured, relational model and by exposing that model both to people (Lightning pages) and to an AI agent (Agentforce) through governed automation.

1.2 Motivation for a CRM-Based Approach

A CRM platform is normally associated with salesand service, but its core capabilities map directly onto logistics tracking: a configurable data model, record-level security, workflow automation, reporting, and an application framework. Building on Salesforce means that authentication, sharing, audit trail, reporting, and user interface components are provided by the platform rather than being re-implemented. This allows the project to concentrate on the logistics-specific parts: the parcel schema, the tracking prompt, the retrieval flow, and the conversational agent. The Developer Edition provides a no-cost environment that includes Prompt Builder, Flow Builder, Agent Builder, and Apex, which makes it suitable for an academic prototype.

1.3 Project Methodology

Theproject followedan incremental, layer-by-layer build. Each layer was verified before the next was added, so that a failure could be traced to a single component. The phases and their deliverables are summarised below.

Phase	Activities	Deliverables
1. Analysis	Identify tracking pain points, define personas and functional requirements.	Requirements FR-01 to FR-12; use cases
2. Data modelling	Create custom objects, fields, auto-number, picklist, and lookups in Object Manager.	Parcel__c, Delivery__c, Sender__c, Receiver__c

Phase	Activities	Deliverables
3. User interface	Create custom tabs and the SwiftShip Tracker Lightning app; review record pages.	Four custom object tabs; app navigation
4. Prompt engineering	Author the Retrieve Parcel Details template with merge tags; preview against test data.	Active prompt template, Version 1
5. Automation	Build the auto-launched flow: Get Records, prompt action, output assignment; run debug.	Parcel Details – V1 flow
6. Agent configuration	Enable Einstein and Agentforce; add the Parcel Tracker subagent and action.	SwiftShip Tracker agent, Version 1
7. Security	Create the Swift Ship permission set; assign it to the Einstein Agent user.	Permission set and assignment
8. Testing	Execute test cases, record observations, plan corrective actions.	Test matrix TC-01 to TC-08; observation log
9. Documentation	Compile this report, diagrams, screenshots, and source listings.	Project report

1.4 Technology Overview

Technology	Role in SwiftShip Tracker	Key capabilities used
Salesforce Developer Edition	Hosting platform and sandbox-like environment for the prototype.	Setup, Object Manager, free development org
Custom Objects	Persistent storage for parcels, deliveries, senders, receivers.	Auto-number, picklists, lookups, field history, reports
Lightning App & Tabs	User navigation across the four objects.	Custom Object Tabs, app navigation items
Prompt Builder	Defines the grounded tracking summary template.	Record merge fields, preview, versions, activation
Flow Builder	Orchestrates retrieval and prompt execution.	Auto-launched flow, Get Records, Action, Assignment, Debug
Agentforce / Agent Builder	Conversational interface and reasoning.	Subagents, actions, preview with trace and grounding check
Apex	Invocable action as a code-based alternative to the flow.	@InvocableMethod, SOQL, WITH USER_MODE, unit tests
Permission Sets	Least-privilege access to objects, fields, and automation.	Object settings, flow access, assignment to agent user

1.5 Assumptions and Constraints

- The prototype runsina singleDeveloper Edition org; no production data is stored.
- Test records are entered manually; there is no integration with courier systems or GPS hardware.
- Parcel IDs are system-generated auto-numbers; users look parcels up by this identifier.
- Generative AI behaviour is bounded by the prompt template and by the permissions of the running user.
- Salesforce may limit or change AI features and may terminate Developer Edition orgs that remain inactive for an extended period; the project should therefore be backed up as metadata.

1.6 Organisation of this Report

Chapter 2 defines requirements and use cases. Chapter 3 presents the architecture. Chapters 4 to 8 describe, in turn, the data model, prompt template, flow, agent, and security design. Chapter 9 documents testing, Chapter 10 evaluates the system, and Chapter 11 sets out the roadmap. Chapter 12 is a step-by-step implementation walkthrough, Chapter 13 is a user guide, Chapter 14 concludes, and Chapter 15 contains the appendices, glossary, and references.

2. Requirements Analysis and System Objectives

Rigorousrequirement definition isfundamentalto platform architecture.In enterprise CRM implementations, formalising functional specifications and quality attributes prevents architectural drift, aligns engineering tasks with operational goals, and ensures that automated workflows directly support business processes. The functional capabilities of the platform were synthesised into twelve primary requirements (FR-01 to FR-12):

ID	Requirement Domain	Operational Description and Impact
FR-01	Record Management	System must support full lifecycle management (creation, update, retrieval) of custom parcel records within Salesforce.
FR-02	Data Identification	System must generate and enforce unique Parcel ID values across all records to guarantee exact indexing and lookup.
FR-03	Lifecycle Tracking	System must maintain core parcel attributes, including status picklist values, package weight, and estimated delivery dates.
FR-04	Stakeholder Management	System must maintain dedicated sender and receiver records linked directly to corresponding parcel records.
FR-05	Delivery Operations	System must support delivery operational tracking, including real-time location metrics and delivery estimate adjustments.
FR-06	Data Retrieval	System must retrieve specific parcel records based on an incoming unique Parcel ID text parameter.
FR-07	Conversational Output	System must generate formatted, natural language conversational tracking updates for customer inquiries.
FR-08	Automation Integration	System must utilise Salesforce Flow Builder to orchestrate data movement between parcel records and Prompt Builder actions.
FR-09	Access Governance	System must enforce granular user access controls to objects, fields, and flows via custom Permission Sets.
FR-10	Interface Navigation	System must provide custom Lightning tabs and structured record layouts to streamline operational navigation.
FR-11	AI Agent Deployment	System must support Agentforce configuration to expose parcel tracking capabilities through conversational AI interfaces.
FR-12	Operational Analytics	System must construct structured custom object data models that serve as the foundation for native Salesforce reports and dashboards.

To complement these capabilities, the architecture enforces non-functional quality attributes across seven dimensions:

- **Usability:** Standard Salesforce Lightning Design System (SLDS) components deliver clear, consistent interfaces and intuitive natural language responses.
- **Security:** Defence-in-depth combining profile constraints, custom permission sets, role hierarchies, and Field-Level Security (FLS).
- **Reliability:** Automation and prompt generation must execute deterministically, ensuring accurate retrieval and zero AI hallucination within platform governor limits.
- **Performance:** Flow execution and Prompt Builder generation must complete within sub-second latency thresholds to support real-time conversational interactions.
- **Scalability:** The data model and low-code flows must accommodate growing transactional volumes, user counts, and high-frequency queries.
- **Maintainability:** Custom objects, prompt templates, flows, and permission sets are built modularly to simplify updates and future enhancements.
- **Data Integrity:** Relational constraints, lookup fields, auto-numbering, and enforced picklist value sets preserve consistency across all lifecycle stages.

2.1 Stakeholders and Actors

Actor	Description	Primary interest
Customer	External person waiting for a parcel.	Quick, accurate status and delivery estimate.
Sender	Person or business that books a parcel.	Confirmation that the parcel is booked and moving.
Delivery Agent	Field courier updating shipment progress.	Simple way to update status and location.
Support Staff	Customer service representative.	Immediate access to parcel, sender, and receiver details.
Delivery Manager	Operations manager.	Oversight of parcels and deliveries; exception handling.
System Administrator	Platform owner.	Configuration, security, and maintainability.
Agentforce Service Agent	Automated agent user (Einstein Agent license).	Permission to run only the actions it needs.

2.2 Use Cases

ID	Use case	Actor	Main flow	Outcome
UC-01	Book a parcel	Support Staff	Create Parcel__c with name, weight, status Booked, estimated delivery, and sender.	Record saved; Parcel ID auto-generated.
UC-02	Update parcel status	Delivery Agent	Open parcel; change Status__c; save.	Status and history updated.
UC-03	Record delivery milestone	Delivery Agent	Create or edit Delivery__c linked to the parcel; set Current_Location__c.	Location visible on delivery record.
UC-04	Look up parcel by ID	Support Staff	Search Parcels tab for Parcel ID.	Parcel record displayed.
UC-05	Ask agent for tracking	Customer / Staff	Type "Track parcel P-001" in the agent.	Natural-language tracking update.
UC-06	Handle unknown ID	Customer / Staff	Ask agent for an ID that does not exist.	"No match" response, no error or invented data.
UC-07	Manage permissions	Administrator	Edit the Swift Ship permission set; assign to users.	Least-privilege access enforced.
UC-08	Review operations	Delivery Manager	Run reports on parcel status counts.	Visibility of workload and delays.

2.3 User Stories and Acceptance Criteria

Story	Acceptance criteria
As a customer, I want to ask for my parcel by its ID so that I know its status without calling support.	Given a valid Parcel ID, the answer shows name, ID, status, weight, and estimated delivery exactly as stored.
As a support agent, I want one place to see parcel, sender, and receiver data so that I can answer inquiries quickly.	Parcel, Sender, and Receiver tabs are available in the SwiftShip Tracker app and respect the user's permissions.
As a delivery agent, I want to update a status picklist so that the parcel history stays current.	Status accepts only Booked, In-Transit, Out for Delivery, or Delivered; changes are tracked in field history.
As an administrator, I want the AI to see only permitted data so that sensitive details are not exposed.	The running agent user can execute only the assigned flow/action and read only permitted fields.
As a manager, I want structured data so that I can build reports on parcel status.	Reports can be created on Parcel__c grouped by Status__c.

2.4 Feasibility Study

Dimension	Assessment
Technical	All required capabilities (custom objects, Prompt Builder, Flow, Agentforce, Apex) are available in the Developer Edition. The team needs only platform administration skills plus basic Apex for the optional action.

Dimension	Assessment
Economic	The prototype uses a free Developer Edition. Production use would require appropriate Salesforce licences and AI usage entitlements; current licensing and pricing must be confirmed with Salesforce before any production plan.
Operational	Support staff already work in a browser; Lightning tabs and an agent chat require minimal training. Data entry discipline (consistent statuses and IDs) is the main operational need.
Schedule	The layered build (data, UI, prompt, flow, agent, security, testing) fits within an academic term, with each layer small enough to verify in a single session.
Legal / Privacy	Sender and receiver contact data is personal data; FLS, least-privilege permissions, and avoidance of real personal data in the prototype address this risk.

2.5 Requirements Traceability Matrix

Thematrix linkseachfunctional requirement to the component that realises it and the test case that verifies it. Requirements without a dedicated test are flagged for future coverage.

Req.	Realised by	Verified by
FR-01 Record management	Parcel__c object, Parcel tab	TC-01
FR-02 Unique Parcel ID	Parcel_ID__c auto-number	TC-01, TC-03
FR-03 Lifecycle attributes	Status__c, Weight__c, Estimated_Delivery__c	TC-02
FR-04 Sender / receiver records	Sender__c, Receiver__c, Parcel lookup	TC-02 (receiver link planned)
FR-05 Delivery operations	Delivery__c with Current_Location__c	Planned TC-10 TC-03
FR-06 Retrieval by Parcel ID	Flow Get Records / Apex SOQL	TC-04
FR-07 Conversational output	Prompt Builder template	TC-05
FR-08 Automation integration	Parcel Details – V1 flow	TC-07
FR-09 Access governance	Swift Ship permission set, FLS	TC-02
FR-10 Interface navigation	Custom tabs, Lightning app	TC-06
FR-11 AI agent deployment	Agentforce Parcel Tracker subagent	Planned TC-13
FR-12 Analytics foundation	Structured custom objects	

3. Enterprise Architectural Framework

Thesystem architecture ofSwiftShip Tracker is structured asa multi-tiered enterprise framework. Decoupling presentation, application logic, data storage, process orchestration, generative AI processing, security governance, and analytics allows individual tiers to be optimised, updated, or expanded without destabilising core schemas or data access security. The eight structural layers are:

1. **User Interaction Layer:** Customer, Sender, Delivery Agent, Support Staff, System Administrator.
2. **Interface Tier:** Salesforce Lightning Experience, Lightning Web Components (LWC), Experience Cloud Digital Experiences.
3. **Application Layer:** SwiftShip Tracker custom application container and operational user workspace.
4. **Data Layer:** Custom Objects (Parcel__c, Delivery__c, Sender__c, Receiver__c).
5. **Automation Tier:** Flow Builder (Auto-Launched Flows), System Actions, Platform Event Notifications.
6. **AI Integration Layer:** Prompt Builder (Retrieve_Parcel_Details), Agentforce Conversational Reasoning Engine.
7. **Security & Governance Layer:** Profiles, Custom Permission Sets, Role Hierarchy, Sharing Rules, FLS.
8. **Analytics Tier:** Salesforce Reports, Operational Dashboards, Executive Analytics.

Data flows downward through this hierarchy during transaction ingestion and upward during query execution. For example, a conversational query submitted at the Interface Tier passes to the AI Integration Layer, triggers the Automation Tier to query the Data Layer under strict Security & Governance enforcement, and returns a grounded response to the user.

Every parcel record transitions through a deterministic sequence of operational states:

Parcel Booking / Record Creation → Parcel Identification → Shipment Processing → In-Transit Tracking → Out for Delivery → Delivered → Customer Notification / Support

- **Parcel Booking / Record Creation:** Initial entry of parcel dimensions, weight, sender, and receiver details into the CRM.
- **Parcel Identification:** System generation of the unique indexing identifier (Parcel_ID__c) used for tracking lookups.
- **Shipment Processing:** Facility sorting, package aggregation, and dispatch preparation.
- **In-Transit Tracking:** Active transport phase, capturing location transitions and route updates on linked Delivery__c records.
- **Out for Delivery:** Final-leg assignment where the parcel is dispatched with field couriers.
- **Delivered:** Verification and confirmation of receipt, terminating active logistics tracking.
- **Customer Notification / Support:** Automated confirmations and AI-assisted conversational support inquiries.



Diagram 1: Eight-layer architecture of SwiftShip Tracker, from user interaction down to analytics.

3.1 Component Interaction

From	To	Interaction	Data exchanged
User	Agentforce	Natural-language request	Free text, e.g. "Track parcel P-001"
Agentforce	Parcel Tracker subagent	Topic classification and routing	Intent, extracted Parcel ID
Subagent	Agent Action	Invocation of flow or Apex	Input variable (Parcel ID)
Action	Parcel__c	Get Records / SOQL	Parcel_ID__c filter; returned record
Action	Prompt Builder	Prompt template execution	Parcel__c record as input context
Prompt Builder	Action	Generated text	promptResponse string
Action	Agentforce	Output variable	Output text
Agentforce	User	Final answer	Grounded tracking summary

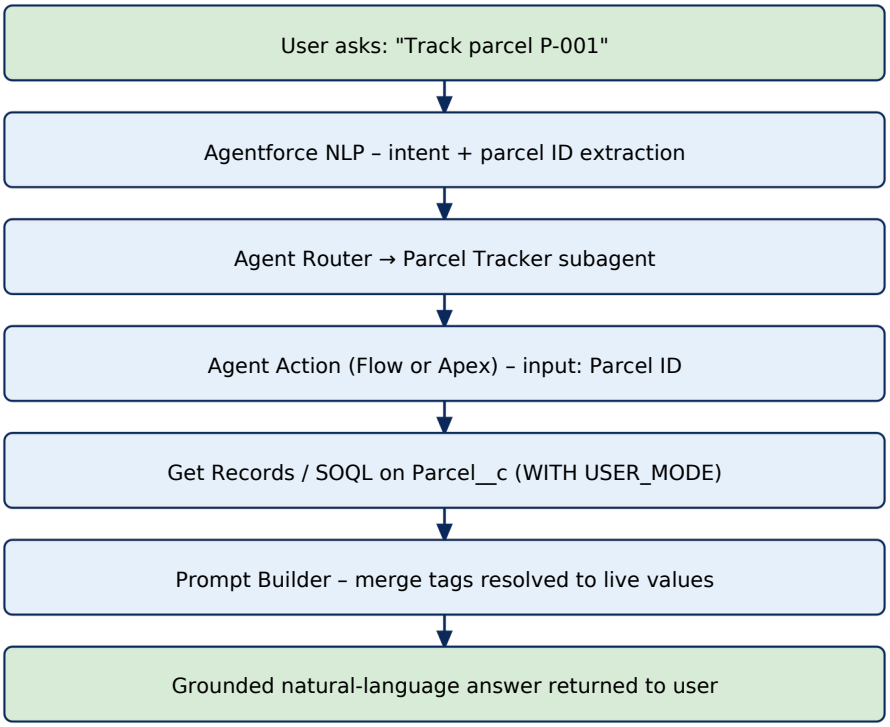


Diagram 2: Request lifecycle for a conversational tracking query.

3.2 Key Design Decisions

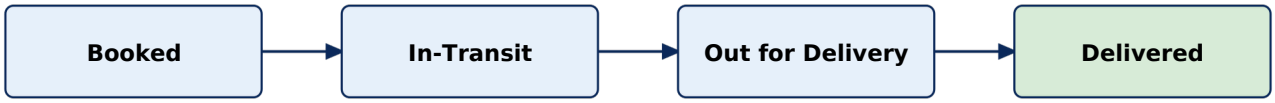
Decision	Alternatives considered	Rationale
Custom objects instead of standard Case or Asset	Standard objects with custom fields	Parcels are the central business entity; dedicated objects give clear naming, tabs, and permissions.
Auto-number Parcel ID (P-{0000})	Manual text IDs; record Id	Guarantees uniqueness and a short, human-friendly lookup key.
Prompt Builder with merge fields	Free-form LLM answer from conversation context	Merge fields bind output to stored values, reducing invented content.
Auto-launched flow as orchestration layer	Apex only; Screen flow	Low-code, reusable by agents, other flows, and scripts; easy to debug visually.
Apex invocable action as optional alternative	Flow only	Provides bulk-safe, testable code path with explicit error handling and WITH USER_MODE.
Permission set for automation access	Profile changes	Permission sets are additive, assignable to the agent user, and easier to audit.

3.3 Deployment and Environment View

Theprototypeisdeployed in oneDeveloper Edition org. Metadata (objects, tabs, prompt template, flow, agent, permission set, Apex classes) lives in that org. For a production path, a recommended promotion model is: Developer Edition or scratch org for build, a Developer or Partial Copy sandbox for integration testing, and a production org with formal release governance. Source control of metadata (for example via Salesforce DX) is recommended so that the project can be restored if the free org is reclaimed.

3.4 Integration Points

- **Inbound:** Users andagents invoke the flow or Apex action; future inbound sources include courier APIs and scanner apps.
- **Outbound:** Future SMS and WhatsApp notifications triggered by status changes through Flow.
- **Internal:** Prompt Builder reads Parcel__c fields; reports read all custom objects.



Status__c picklist values – deterministic forward progression

Diagram 3: Parcel lifecycle states represented by the Status__c picklist.

4. Data Architecture and Salesforce Custom Schema

Thedata architecture is implemented using custom Salesforceobjects configuredwithin Object Manager. Normalised relational object design preserves data integrity, models enterprise business relationships, and exposes reliable record context to AI prompt grounding engines. Isolating logistics entities into discrete custom objects eliminates redundancy and enforces strict normalisation. The schema comprises four core custom objects:

Object	API Name	Role	Functional Description & Core Fields
Parcel	Parcel__c	Primary Operational Entity	Core shipment record holding tracking parameters, physical attributes, status, and lookup links. Fields: Name (Text), Parcel_ID__c (Auto-Number), Status__c (Picklist), Weight__c (Number), Estimated_Delivery_Date__c (Date), Sender__c (Lookup).
Delivery	Delivery__c	Logistics Tracking Entity	Active delivery tracking data and location milestones. Fields: Current_Location__c (Text), Estimated_Delivery__c (Date), Parcel__c (Lookup), Sender__c (Lookup).
Sender	Sender__c	Stakeholder Origin Entity	Origin party contact attributes and address details. Fields: Sender_Name__c (Text), Address__c (Text/TextArea), Contact__c (Phone), Email__c (Email).
Receiver	Receiver__c	Stakeholder Destination Entity	Destination party attributes, contact information, and final delivery locations. Fields: Receiver_Name__c (Text), Address__c (Text/TextArea), Contact__c (Phone), Email__c (Email).

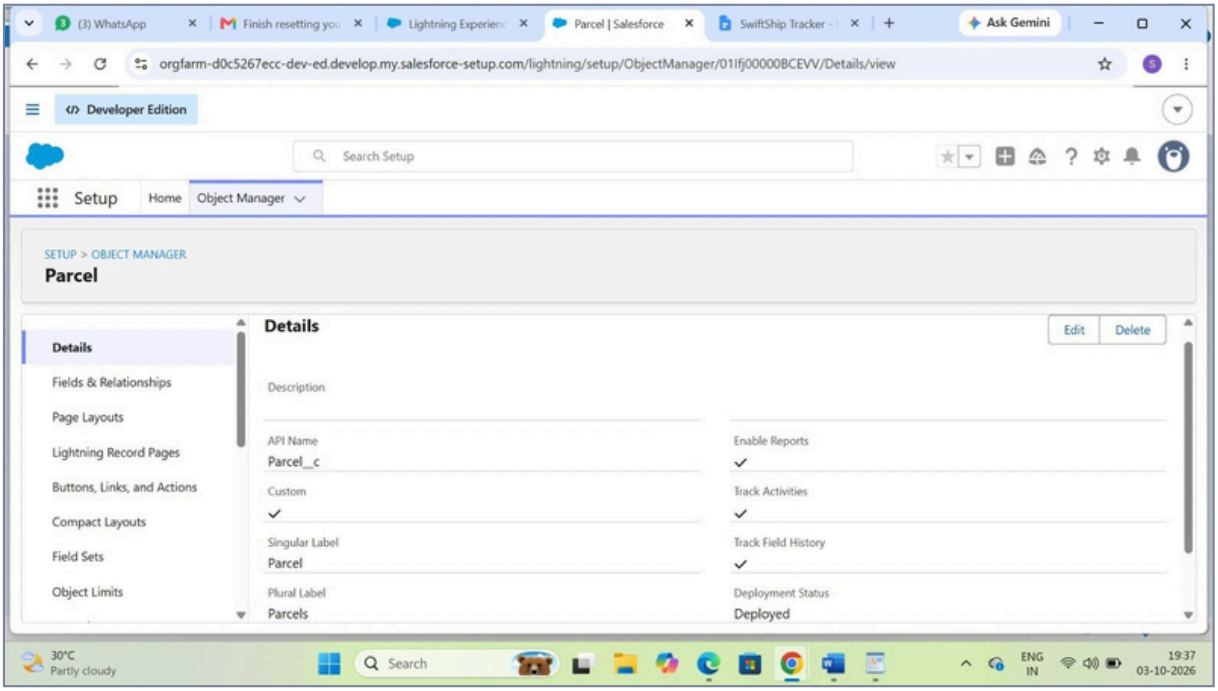


Figure 1: Parcel (Parcel__c) custom object details in Salesforce Object Manager – API name Parcel__c, Custom, Enable Reports, Track Activities, Track Field History, Deployment Status: Deployed.

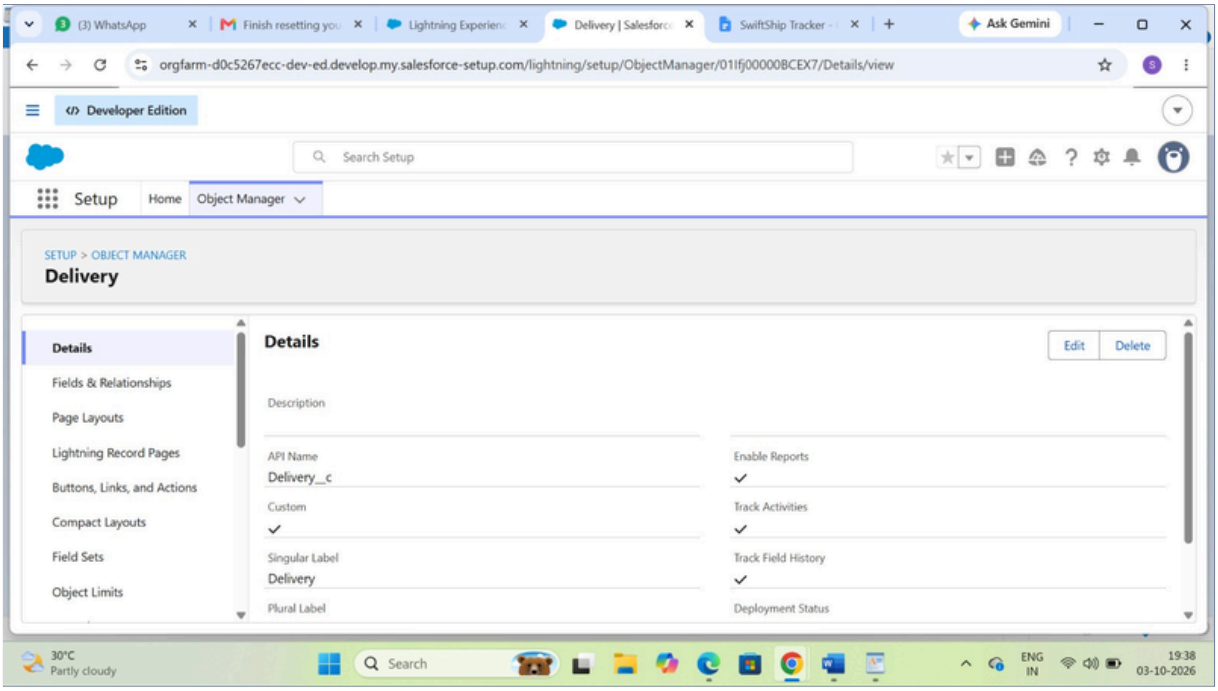


Figure 2: Delivery (Delivery__c) custom object details in Object Manager.

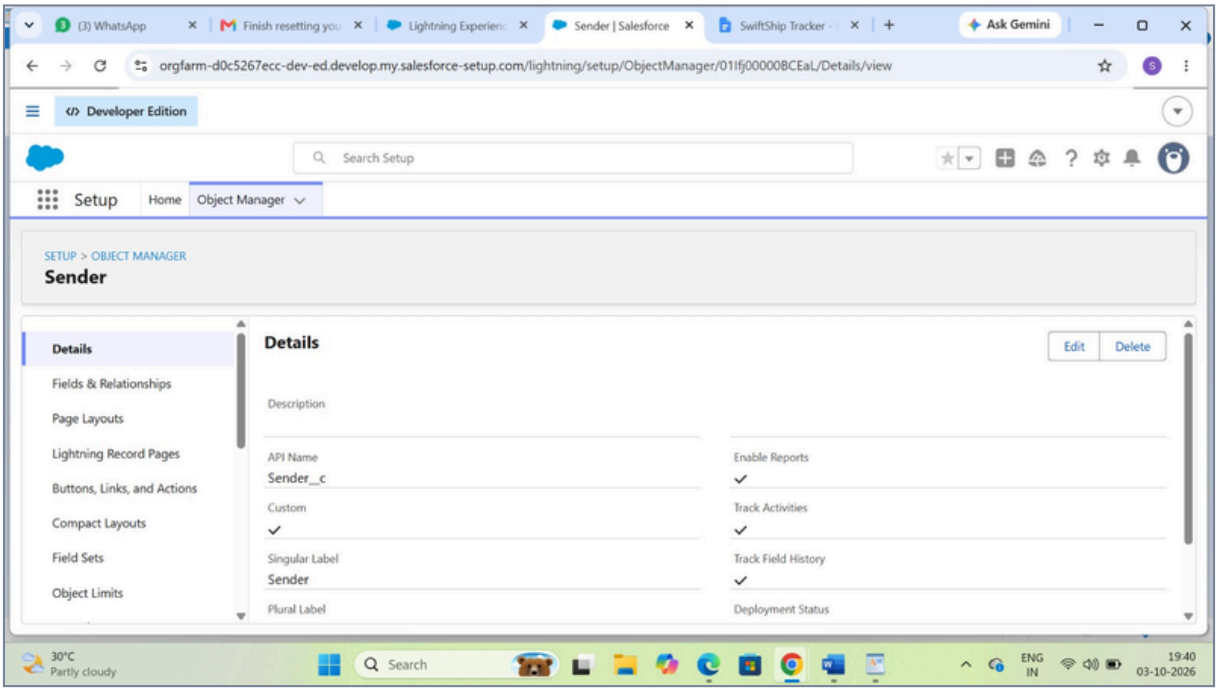


Figure 3: Sender (Sender__c) custom object details in Object Manager.

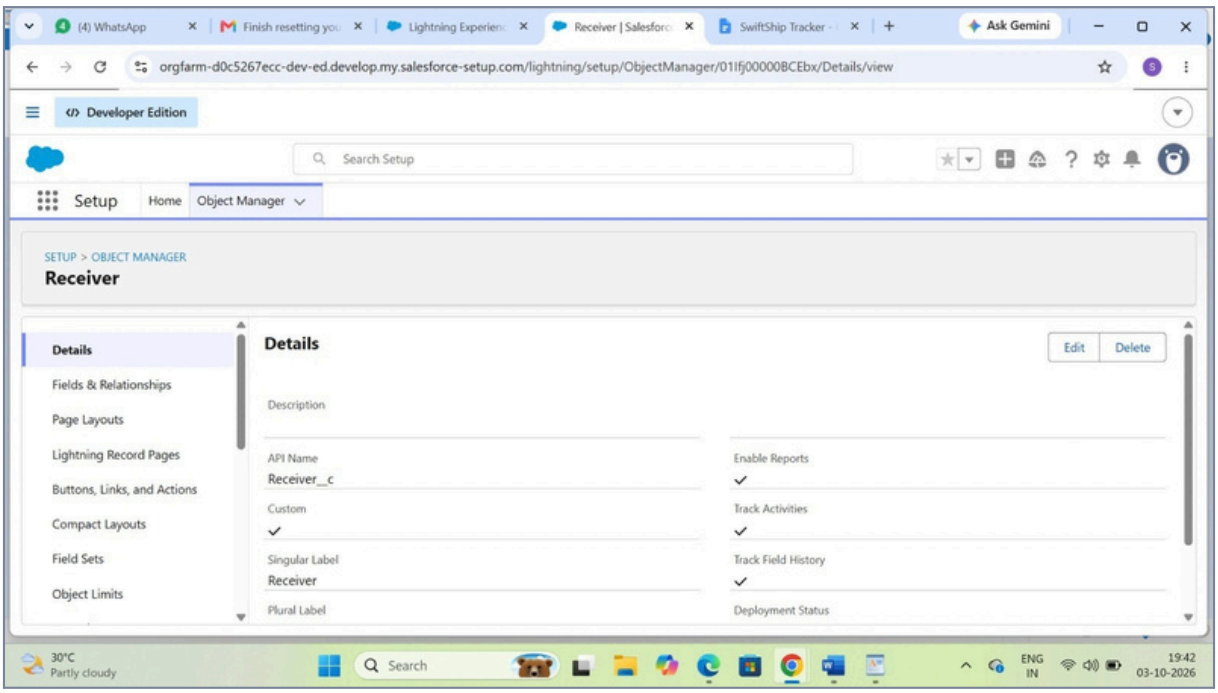


Figure 4: Receiver (Receiver__c) custom object details in Object Manager.

The field-level specification for the central operational entity, Parcel__c, is as follows:

Field Label	API Name / Data Type	Constraint	Operational Purpose
Parcel Name	Name (Text)	Text(80)	Primary display label of the package record (e.g., Laptop, Sandwood).
Parcel ID	Parcel_ID__c (Auto-Number)	Unique indexed (P-{0000})	System-generated tracking ID formatted as P-001, P-002 ensuring indexed key retrieval.
Status	Status__c (Picklist)	Booked, In-Transit, Out for Delivery, Delivered	Current lifecycle state for routing, filtering, and customer support.
Weight	Weight__c (Number)	Number(16, 2)	Physical package mass in kilograms for routing and capacity planning.
Estimated Delivery	Estimated_Delivery_Date__c (Date)	Standard Date field	Target delivery date for milestone management and AI grounding.
Sender	Sender__c (Lookup)	Lookup(Sender__c)	Connects the package to the origin stakeholder profile.

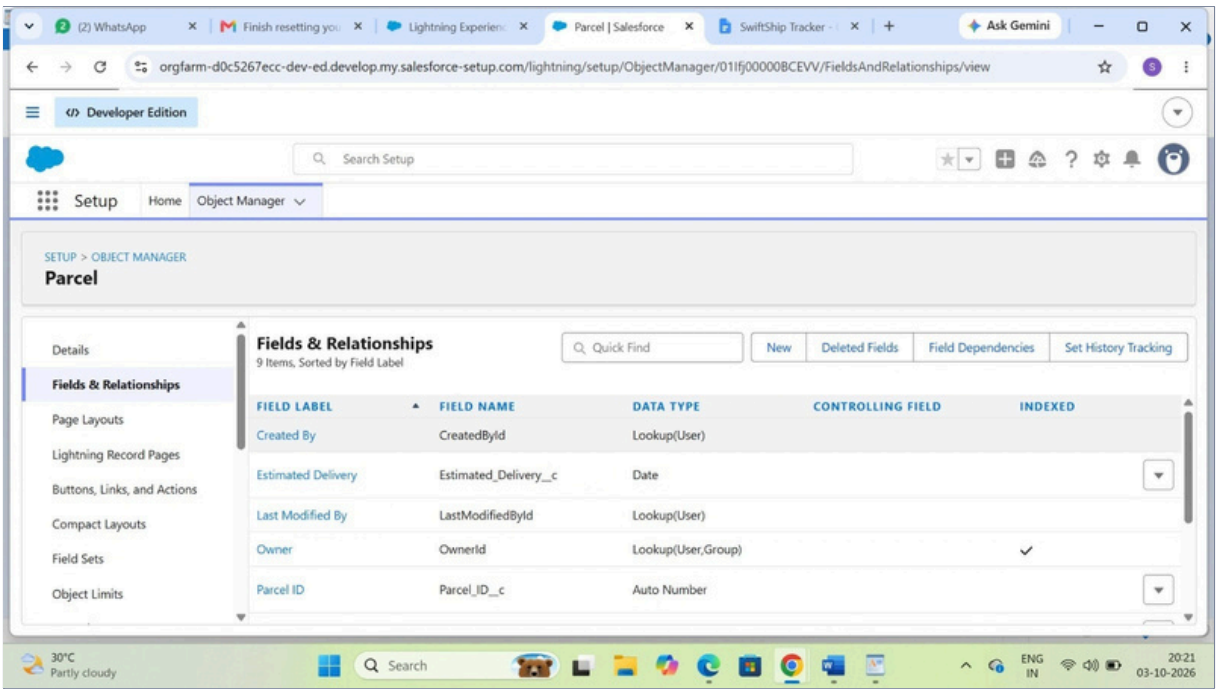


Figure 5: Fields & Relationships view of the Parcel object (9 items), showing Estimated Delivery (Date) and Parcel ID (Auto Number) among the custom fields.

To expose these objects to operational staff, logistics managers, and customer service teams, a custom navigation strategy was executed in Salesforce Setup. Dedicated Custom Object Tabs were created for Parcel__c, Delivery__c, Sender__c, and Receiver__c and configured within the custom "SwiftShip Tracker" Lightning Application, allowing users to navigate shipment profiles, update status picklists, manage stakeholder accounts, and view operational records without complex navigation overhead.

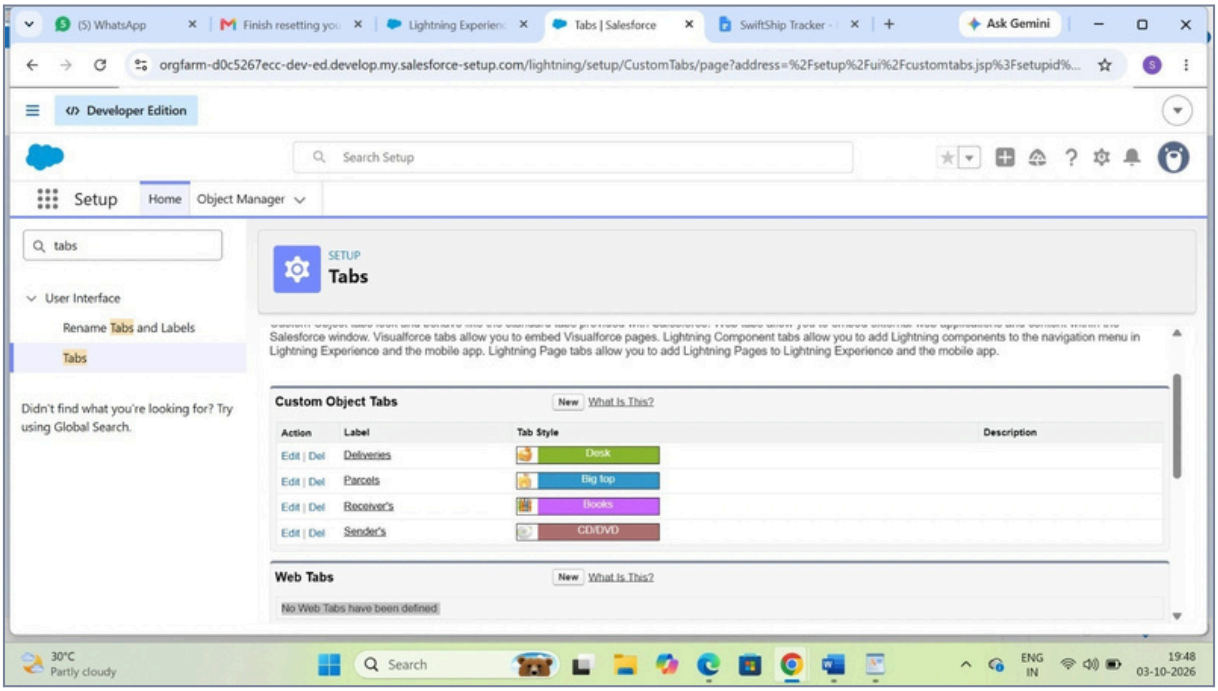


Figure 6: Custom Object Tabs configured in Setup > Tabs: Deliveries, Parcels, Receiver's, and Sender's.

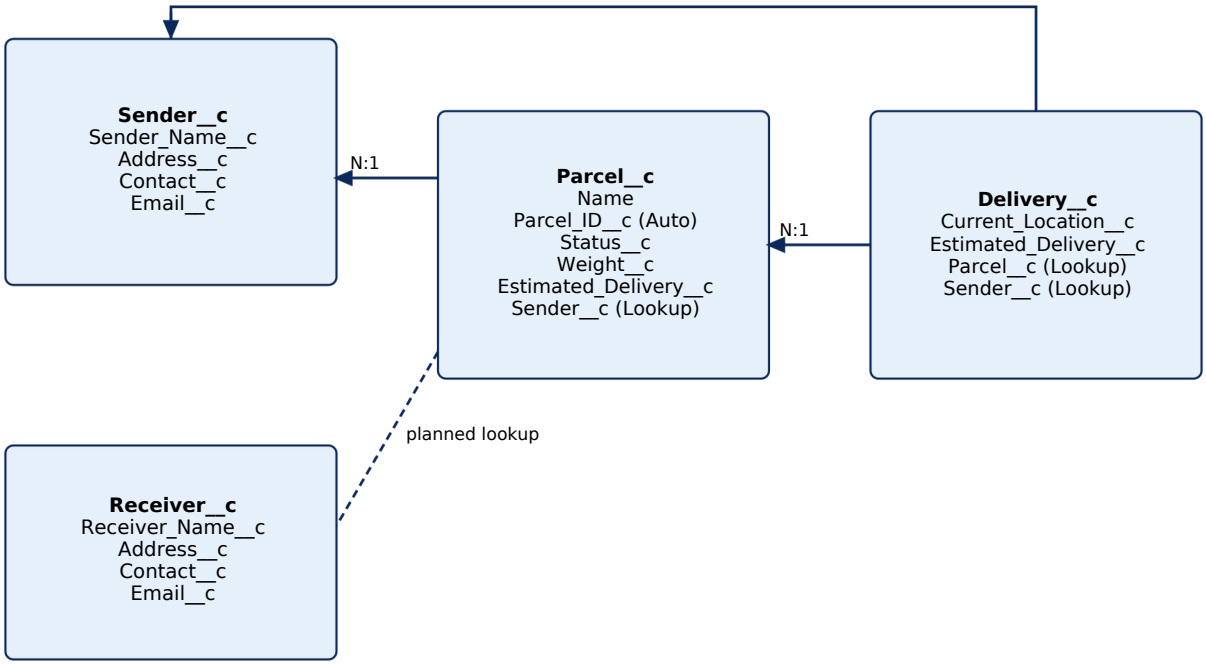


Diagram 4: Entity-relationship view of the SwiftShip Tracker schema. The Receiver link is shown dashed because it is a recommended addition.

4.1 Data Dictionary

Thetables belowlistthefields designed for each supporting object. Parcel__c is described in the preceding section.

Object	Field	Data type	Purpose
Delivery__c	Name	Text / Auto-number	Delivery record label.
Delivery__c	Current_Location__c	Text	Latest known location of the parcel.
Delivery__c	Estimated_Delivery__c	Date	Adjustable delivery estimate for this shipment leg.
Delivery__c	Parcel__c	Lookup(Parcel__c)	Links the delivery to its parcel.
Delivery__c	Sender__c	Lookup(Sender__c)	Links the delivery to the origin party.
Sender__c	Sender_Name__c	Text	Name of the sending person or business.
Sender__c	Address__c	Text / Text Area	Origin address.
Sender__c	Contact__c	Phone	Contact number (PII – protect with FLS).
Sender__c	Email__c	Email	Contact email (PII – protect with FLS).
Receiver__c	Receiver_Name__c	Text	Name of the receiving person or business.
Receiver__c	Address__c	Text / Text Area	Destination address.
Receiver__c	Contact__c	Phone	Contact number (PII – protect with FLS).
Receiver__c	Email__c	Email	Contact email (PII – protect with FLS).

4.2 Relationships

Child	Parent	Type	Delete behaviour (recommended)
Parcel__c	Sender__c	Lookup (N:1)	Clear the field if the sender is deleted; do not cascade-delete parcels.
Delivery__c	Parcel__c	Lookup (N:1)	Restrict deletion of a parcel that has deliveries.
Delivery__c	Sender__c	Lookup (N:1)	Clear the field if the sender is deleted.

Child	Parent	Type	Delete behaviour (recommended)
Parcel__c	Receiver__c	Planned lookup (N:1)	Add a Receiver__c lookup on Parcel__c to satisfy FR-04 fully.

Observation: requirement FR-04 asks for sender and receiver records linked to parcels, but the current Parcel__c field list contains only the Sender lookup. Adding a Receiver lookup is a small change and is recommended before any production use.

4.3 Status Semantics and Transition Rules

Status	Meaning	Allowed next status
Booked	Parcel record created and identified; not yet dispatched.	In-Transit
In-Transit	Parcel is moving between facilities.	Out for Delivery
Out for Delivery	Parcel is with the courier for final delivery.	Delivered (or back to In-Transit if the attempt is rerouted)
Delivered	Receipt confirmed; tracking ends.	None

4.4 Recommended Validation Rules

Validation rules protect data integrity at entry time. The formulas below are recommended additions; they use the field API names from the schema.

Validation rule definitions (Parcel__c)

```
Rule: Weight_Must_Be_Positive
Formula:    AND(NOT(ISBLANK(Weight__c)), Weight__c <= 0)
Message:    Weight must be greater than zero.

Rule: Estimate_Not_In_Past_On_Create
Formula:    AND(ISNEW(), Estimated_Delivery__c < TODAY())
Message:    Estimated delivery cannot be in the past for a new parcel.

Rule: No_Status_Regression_From_Delivered
Formula:    AND(ISCHANGED(Status__c), PRIORVALUE(Status__c) = 'Delivered')
Message:    A delivered parcel cannot change status.
```

4.5 Test Data Design

Parcel name	Parcel ID	Status	Weight	Estimated delivery	Purpose
Laptop	P-001	Booked	2	10/17/2026	Primary test record used in prompt, flow, and debug runs.
Sandwood	P-002	As entered in org	As entered	As entered	Secondary record to verify auto-number indexing and lookup binding.
(none)	P-9999999	n/a	n/a	n/a	Non-existent ID for the "no match" edge case (TC-08).

4.6 Example SOQL Queries

Representative SOQL

```
// Retrieve one parcel by its Parcel ID
SELECT Name, Parcel_ID__c, Status__c, Weight__c, Estimated_Delivery__c
FROM Parcel__c
WHERE Parcel_ID__c = 'P-001'
LIMIT 1

// Parcel count by status (basis for a dashboard)
SELECT Status__c, COUNT(Id) total
FROM Parcel__c
GROUP BY Status__c

// Deliveries and their parcels
SELECT Name, Current_Location__c, Parcel__r.Name, Parcel__r.Status__c
FROM Delivery__c
ORDER BY CreatedDate DESC
```

4.7 Page Layouts and Record Pages

Each custom object hasa default page layout and a Lightning record page. For Parcel__c the highlights panel and details section present Parcel ID, Status, Weight, and Estimated Delivery together so that support staff see the tracking essentials without scrolling. Related lists on the parcel page show linked deliveries. The Track Activities and Track Field History options are enabled on the objects (Figures 1 to 4), so status changes and tasks are available for audit and follow-up.

5. Conversational AI Integration: Prompt Builder Mechanics

Deploying generativeAIin enterprise logistics requires strictdynamic grounding.Standardlarge language models operating without context can hallucinate shipment states, misinforming customers. SwiftShip Tracker mitigates this by using Salesforce Prompt Builder to bind prompt templates directly to live CRM record fields, so that AI responses reflect database truth. The primary generative asset is the "Retrieve Parcel Details" prompt template:

Configuration Parameter	Technical Specification / Value
Template Name	Retrieve Parcel Details
API Name	Retrieve_Parcel_Details
Functional Purpose	Format underlying Parcel__c database fields into natural language tracking summaries.
Input Context	Parcel__c Record Context / Passed Parcel_ID__c Filter Parameter
Output Format	Formatted Natural Language Parcel Tracking Update

The dynamic grounding expression configured in the template uses exact record merge tags:

Parcel Tracking Update
- Parcel Name: {!$Input:Parcel__c.Name}
- Parcel ID: {!$Input:Parcel__c.Parcel_ID__c}
- Status: {!$Input:Parcel__c.Status__c}
- Weight: {!$Input:Parcel__c.Weight__c}
- Estimated Delivery Date: {!$Input:Parcel__c.Estimated_Delivery_Date__c}

At runtime the merge tags resolve against live record values: Name → package label (e.g., Laptop); Parcel_ID__c → unique identifier (e.g., P-001); Status__c → current picklist state (e.g., Booked); Weight__c → numeric weight (e.g., 2); Estimated_Delivery_Date__c → target delivery date (e.g., 10/17/2026). When invoked, Prompt Builder ingests the specific Parcel__c record, substitutes field values into the merge tags, and returns a grounded textual tracking summary.

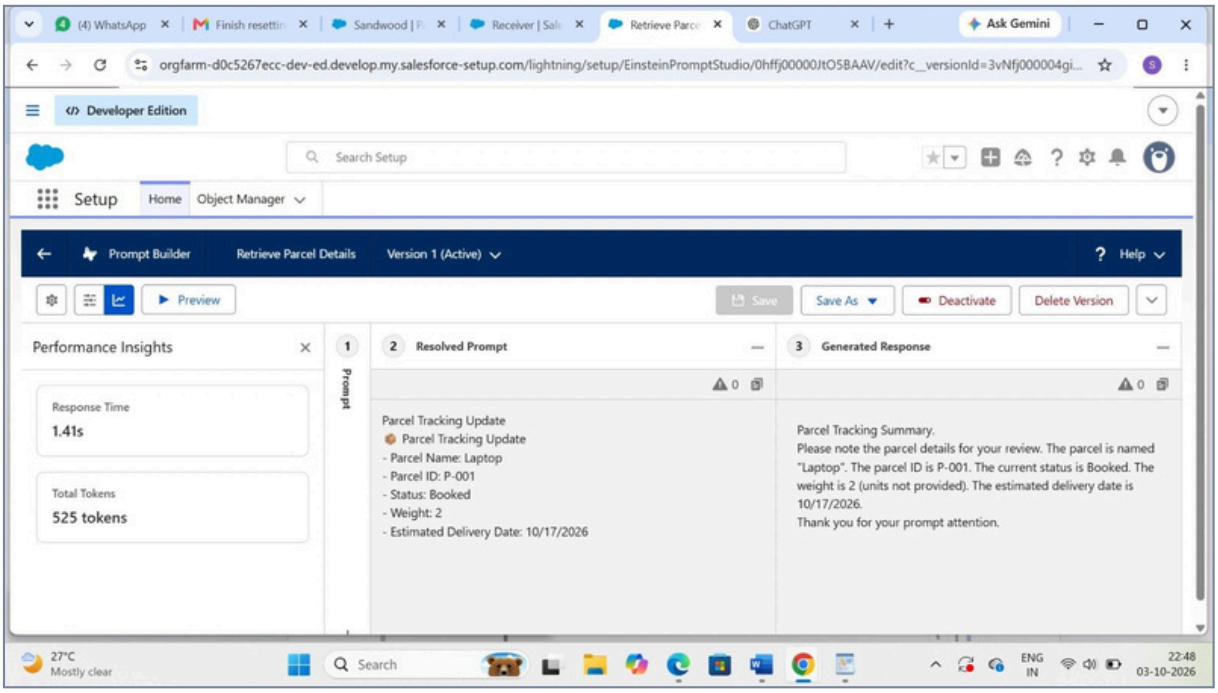


Figure 7: Prompt Builder – "Retrieve Parcel Details" (Version 1, Active). The Resolved Prompt panel shows merged values for record P-001 (Laptop, Booked, weight 2, estimated delivery 10/17/2026) and the Generated Response panel shows the resulting tracking summary (response time 1.41 s, 525 tokens).

5.1 Prompt Design Principles

- **Ground every statement in a field:** each fact in the answer comes from a merge field, not from model memory.
- **Keep the structure fixed:** a labelled list (name, ID, status, weight, estimated delivery) makes output predictable and easy to verify.
- **State the role and constraints:** the model should format the supplied data, not add information.
- **Handle missing values explicitly:** if a field is empty the answer should say it is not available rather than guess.
- **Version and test:** activate a version only after previewing it against known records.

5.2 Resolution Example for Record P-001

Merge field		Rendered in output
{!$Input:Parcel__c.Name}		Parcel Name: Laptop
{!$Input:Parcel__c.Parcel_ID__c}		Parcel ID: P-001
{!$Input:Parcel__c.Status__c}		Status: Booked
{!$Input:Parcel__c.Weight__c}		Weight: 2
{!$Input:Parcel__c.Estimated_Delivery_Date__c}		Estimated Delivery Date: 10/17/2026

5.3 Observations from Prompt Testing

Preview testing in Prompt Builder (Figure 7) produced a response in 1.41 seconds using 525 tokens. Comparing the Prompt Builder preview with the flow debug output (Figure 9) for the same record reveals three useful points:

- **Data values are consistent.** Name, ID, status, weight, and date are identical in both outputs.
- **Wording varies between runs.** The preview said the weight was "2 (units not provided)", whereas the flow run said "2 kg". Generative models can phrase the same facts differently, so "zero hallucination" is best understood as faithfulness of the data values to the record, not identical wording.
- **Units are not part of the stored value.** Weight__c holds a number only, so the unit must be supplied by the template.

5.4 Recommended Template Refinement

The refined template below fixes the unit, constrains the model to the supplied data, and defines behaviour for blank fields. It keeps the original merge tags.

Refined Retrieve_Parcel_Details template (recommended)

```
You are a parcel tracking assistant. Use ONLY the data below.
Do not add facts that are not listed. If a value is blank, write "not available".
Write a short, polite tracking update in plain text.

Parcel Tracking Update
- Parcel Name: {!$Input:Parcel__c.Name}
- Parcel ID: {!$Input:Parcel__c.Parcel_ID__c}
- Status: {!$Input:Parcel__c.Status__c}
- Weight: {!$Input:Parcel__c.Weight__c} kg
- Estimated Delivery Date: {!$Input:Parcel__c.Estimated_Delivery_Date__c}
```

5.5 Trust, Grounding and Limits

Salesforce applies its Einstein Trust Layer to prompts sent through Prompt Builder, which includes protections such as masking of sensitive data and controls on how prompt data is retained by model providers. The exact behaviours depend on the org configuration and current platform release, so the administrator should confirm them in Salesforce documentation. Independently of the Trust Layer, SwiftShip Tracker limits risk through its design: the prompt receives a single record, the record is fetched under the running user's permissions, and the template forbids additional facts.

6. Process Automation Architecture: Flow Builder Orchestration

Salesforce Flow Builder acts asthe orchestration middle layer.While Prompt Builder defines the format of AI output, an automated process is needed to receive incoming queries, execute database lookups, pass record context to the prompt template, and route the generated response back to consumers. The auto-launched flow "Parcel Details Flow" (Parcel Details – V1) performs this role. Its interface variables are:

Resource Name	Type	Functional Role
ids	Text Variable (Input)	Receives the target Parcel_ID__c string passed from external callers, scripts, or Agentforce actions.
Output	Text Variable (Output)	Captures and exposes the final natural language tracking response returned by Prompt Builder.

The execution logic is a single-path workflow (Start → Get Records → Action → Assignment → End) proceeding across five steps:

1. **Flow Invocation & Triggering:** The flow is invoked by an external agent or process, passing an input string (e.g., 'P-001') into ids.
2. **Data Ingestion (Get Records):** The flow queries Parcel__c for a record where Parcel_ID__c equals the incoming ids value.
3. **AI Action Invocation (Action):** The flow calls the Prompt Builder action Retrieve_Parcel_Details, passing the retrieved record as context.
4. **Variable Assignment (Assignment):** The generated text is assigned to the output variable (Output = {!Parcel_Updates.promptResponse}).
5. **Termination & Payload Return:** Execution completes and the populated Output is returned to the calling agent or interface.

The Assignment element acts as an integration gateway, mapping internal AI prompt results into standard, consumable flow variables. This decouples AI generation logic from front-end presentation channels, allowing any calling system to request tracking updates cleanly.

Flow resources and assignment (Parcel Details – V1)
Input variable: ids (Text, Available for input)
Get Records: Object = Parcel__c, Condition: Parcel_ID__c Equals {!ids}
Action: Parcel Updates (Prompt Template: Retrieve_Parcel_Details)
Assignment: {!Output} Equals {!Parcel_Updates.promptResponse}
Output variable: Output (Text, Available for output)

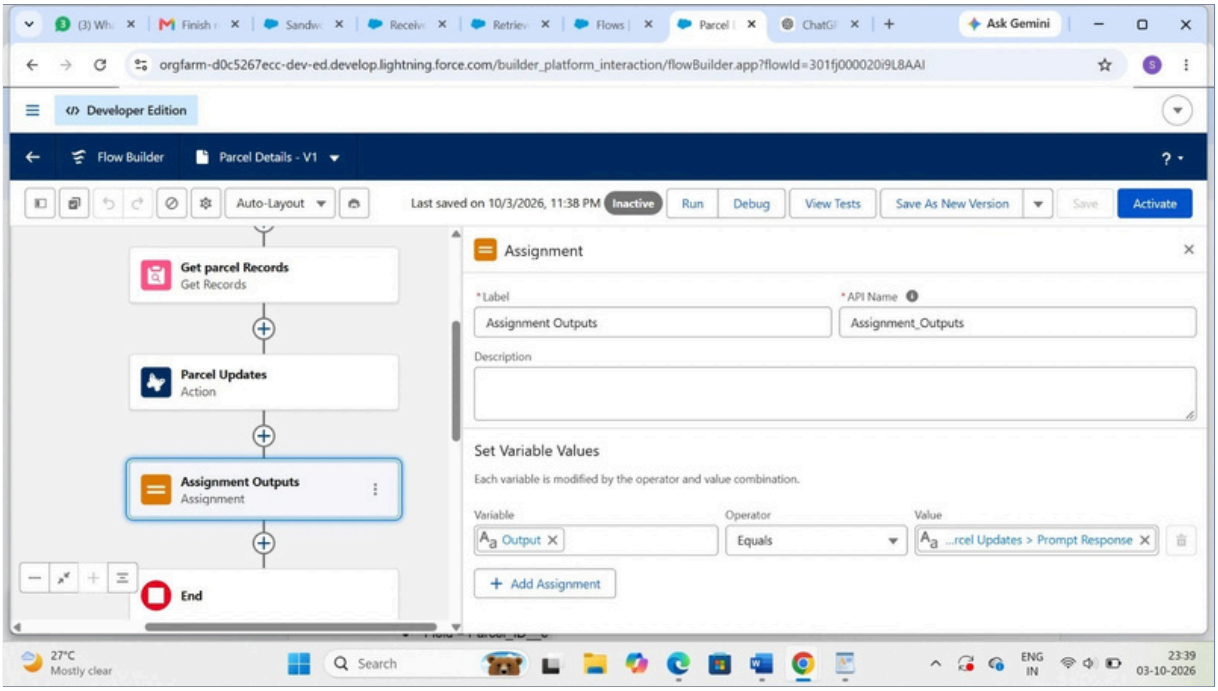


Figure 8: Flow Builder – "Parcel Details – V1": Get parcel Records → Parcel Updates (Prompt Builder action) → Assignment Outputs, where the variable Output equals the Parcel Updates Prompt Response.

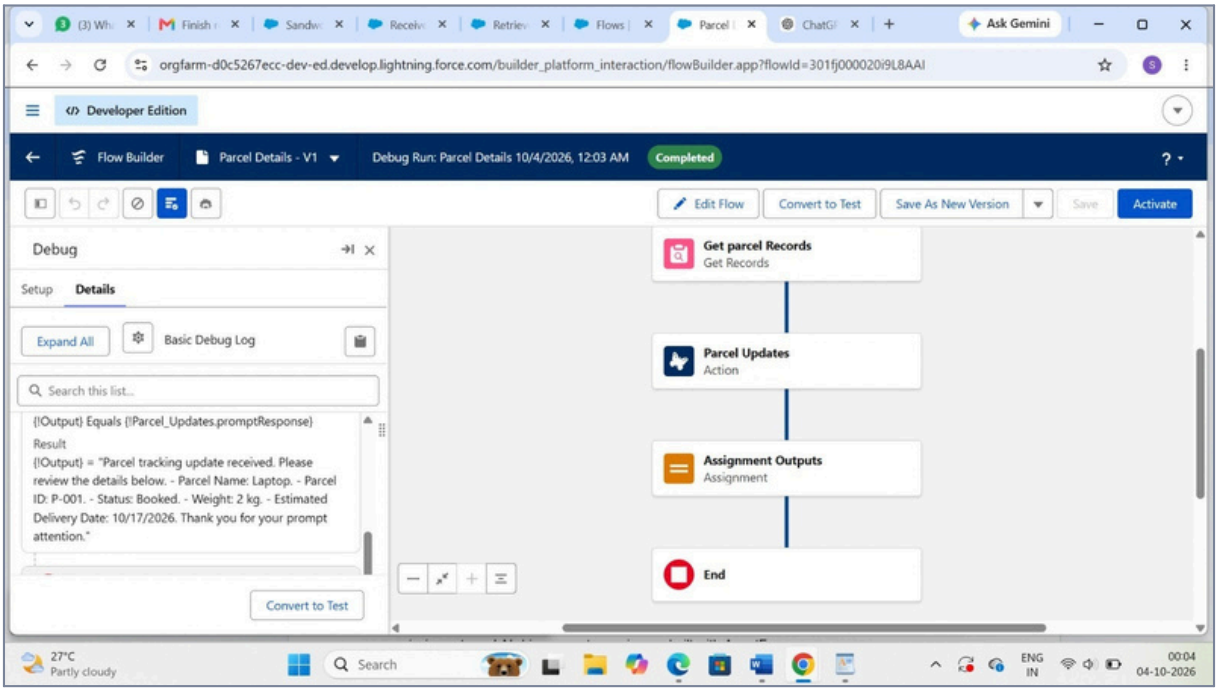


Figure 9: Flow debug run (Completed) – the Output variable resolves to the tracking text for Parcel Name: Laptop, Parcel ID: P-001, Status: Booked, Weight: 2 kg, Estimated Delivery Date: 10/17/2026.

6.1 Flow Element Reference

Element	Type	Configuration	Purpose
Start	Auto-launched	No trigger; invoked by agent, Apex, or another flow	Entry point
Get parcel Records	Get Records	Object Parcel__c; Parcel_ID__c equals {!ids}; first record only	Retrieve the matching parcel
Parcel Updates	Action	Prompt template Retrieve_Parcel_Details; input = record from Get Records	Generate tracking text
Assignment Outputs	Assignment	{!Output} Equals {!Parcel_Updates.promptResponse}	Expose text as output variable

Element	Type	Configuration	Purpose
End	End	Returns Output to caller	Terminate flow

6.2 Fault Handling Design

The currentflow follows a single success path. Two additions are recommended before production use. First, a Decision element after Get Records should test whether a record was found; if not, the flow assigns a fixed message such as "No parcel found for the supplied ID" to Output and ends, which implements the "no match" handling expected by TC-08. Second, the Action element should have a Fault connector that assigns a safe error message and logs the fault so that a prompt or permission failure never surfaces as a raw platform error.

Path	Condition	Output assigned
Success	Record found and prompt succeeds	Prompt response text
No match	Get Records returns no record	"No parcel found for the supplied ID."
Fault	Prompt action or lookup throws an error	"Tracking is temporarily unavailable. Please try again."

6.3 Debugging and Observed Results

Thedebug run (Figure 9) completedsuccessfully and showed the output assignment, confirming that {!Output} was populated from {!Parcel_Updates.promptResponse}. When debugging, set the input variable ids to a known Parcel ID such as P-001, expand the debug log to view each element's inputs and outputs, and confirm that Get Records found exactly one record before inspecting the prompt result.

Activation note: the flow shown in Figure 8 has status Inactive. A flow must be activated before it can be used as an agent action, so activation is a required deployment step.

6.4 Flow versus Apex Action

Criterion	Flow action	Apex invocable action
Build effort	Low – declarative	Medium – code and tests
Bulk handling	Platform-managed	Explicitly coded; single SOQL for all inputs
Error control	Fault paths and decisions	Try/catch, structured result fields
Security	Run mode setting	WITH USER_MODE enforces object and field security
Testing	Flow tests and debug	Apex unit tests with assertions
Maintainability	Visual and admin-friendly	Developer-friendly; version control friendly

6.5 Governor Limits Relevant to the Design

Limit (synchronous Apex/Flow)	Value	Impact on this design
SOQL queries per transaction	100	One query per invocation batch – far below the limit
Records retrieved by SOQL	50,000	Single parcel per request
DML statements	150	No DML in the tracking path
CPU time	10,000 ms	Lookup and prompt call use minimal CPU
Heap size	6 MB	Single record and short text payload

Limits are published by Salesforce and can change between releases; verify the current values in the Salesforce Developer documentation.

7. Conversational Agent Configuration: Agentforce Deployment

DeployingAgentforcetransforms standard CRM datalookupsinto an autonomous conversational experience. By exposing low-code flows to conversational agents, staff and customers can query tracking details in natural language and receive instant, factual responses without manual support intervention. The following platform prerequisites were configured in Setup:

- **Einstein AI Enablement:** Core Einstein platform services activated in the Developer Edition instance.
- **Agentforce Service Activation:** Runtime communication channels and Agentforce services initialised.
- **Prompt Builder Feature Integration:** Prompt template permissions and dynamic grounding options enabled.
- **Agent Builder Workspace Setup:** The Agentforce Default Agent configured in Agent Builder to host topics and actions.

Connecting the auto-launched flow to the conversational interface involves creating custom Agent Actions in Agent Builder:

1. **Workspace Access:** Open Agent Builder and select the parcel-tracking subagent.
2. **Action Initialisation:** Create a new Agent Action with "Flow" as the execution source.
3. **Flow Binding:** Select the auto-launched Parcel Details Flow (Parcel Details – V1).
4. **Parameter Mapping:** Map extracted parcel identifiers to the flow input ids, and bind the flow output Output to the response string.
5. **Topic Association:** Attach the action to the active parcel-tracking subagent.
6. **Agent Activation:** Save, test, and activate the Agentforce workspace.

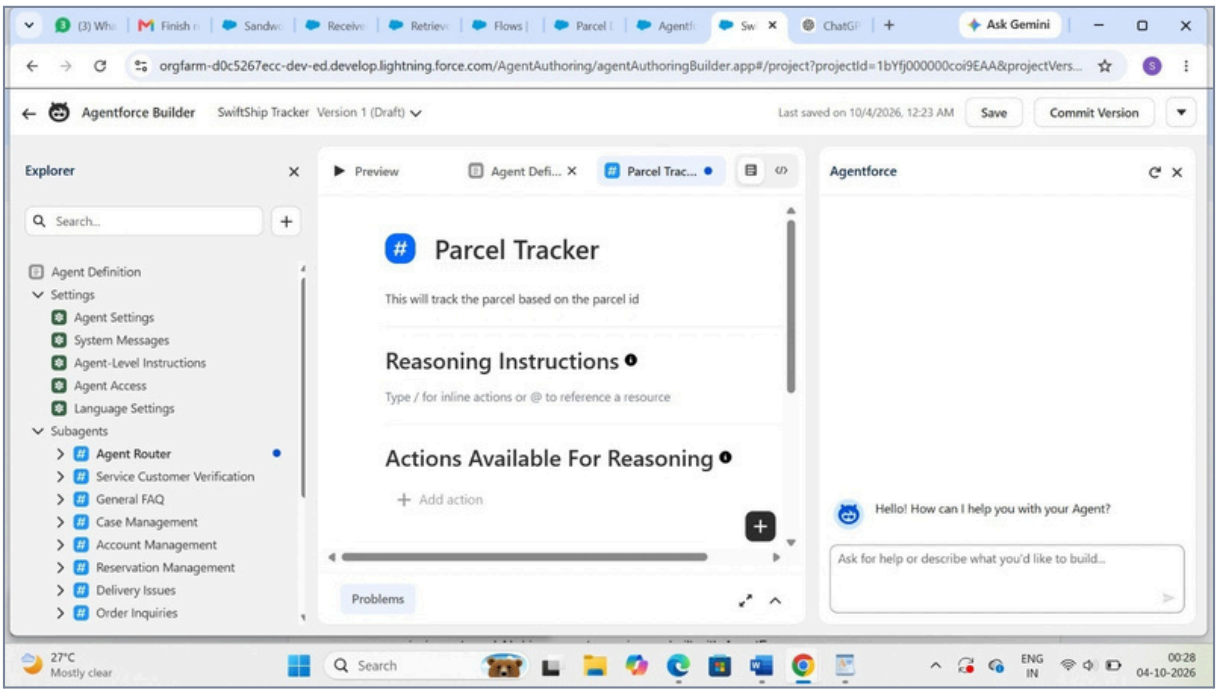


Figure 10: Agentforce Builder – "SwiftShip Tracker" agent (Version 1, Draft) with the new "Parcel Tracker" subagent (description: tracks the parcel based on the parcel ID) added alongside the standard subagents.

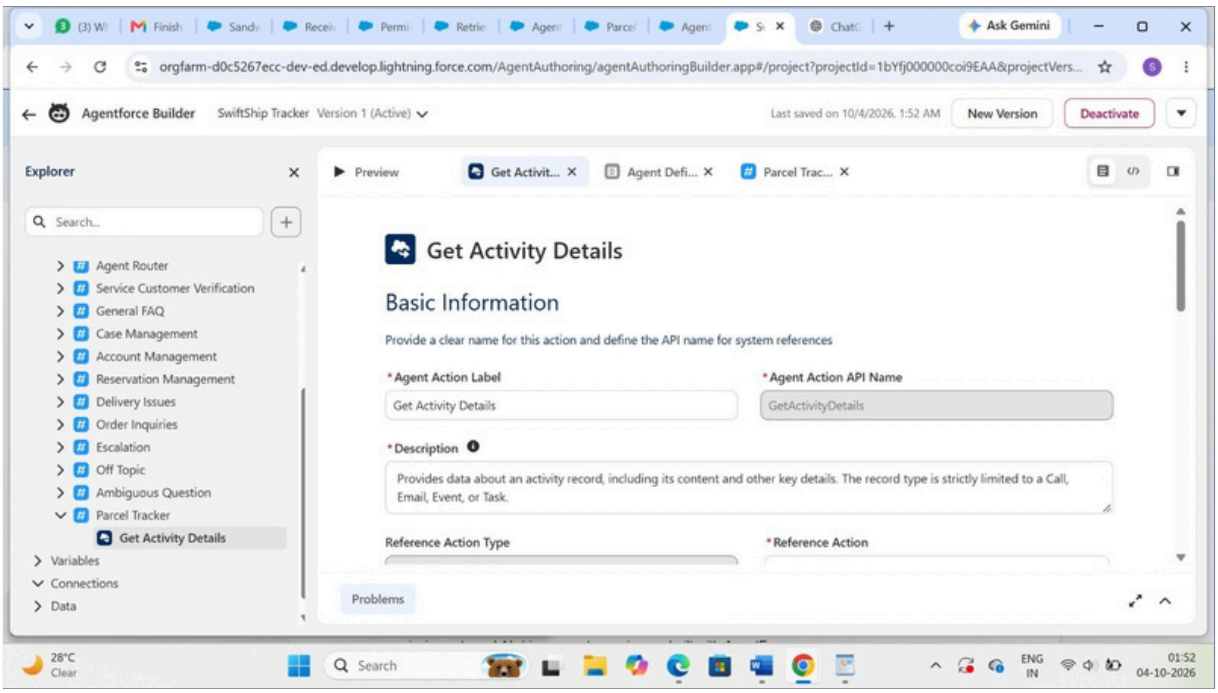


Figure 11: Agentforce Builder – SwiftShip Tracker (Version 1, Active). The Parcel Tracker subagent is expanded and an action ("Get Activity Details") is attached under it.

When a user submits an unstructured inquiry such as "Track parcel P-001", the Agentforce reasoning engine executes a structured cycle: (1) NLP parses the intent and isolates the filter parameter (ids = 'P-001'); (2) the agent invokes the custom action bound to the Parcel Details Flow; (3) the flow runs its Get Records node; (4) the flow invokes Prompt Builder, whose tags resolve against database values; (5) the flow populates the Output variable and Agentforce presents the natural language response.

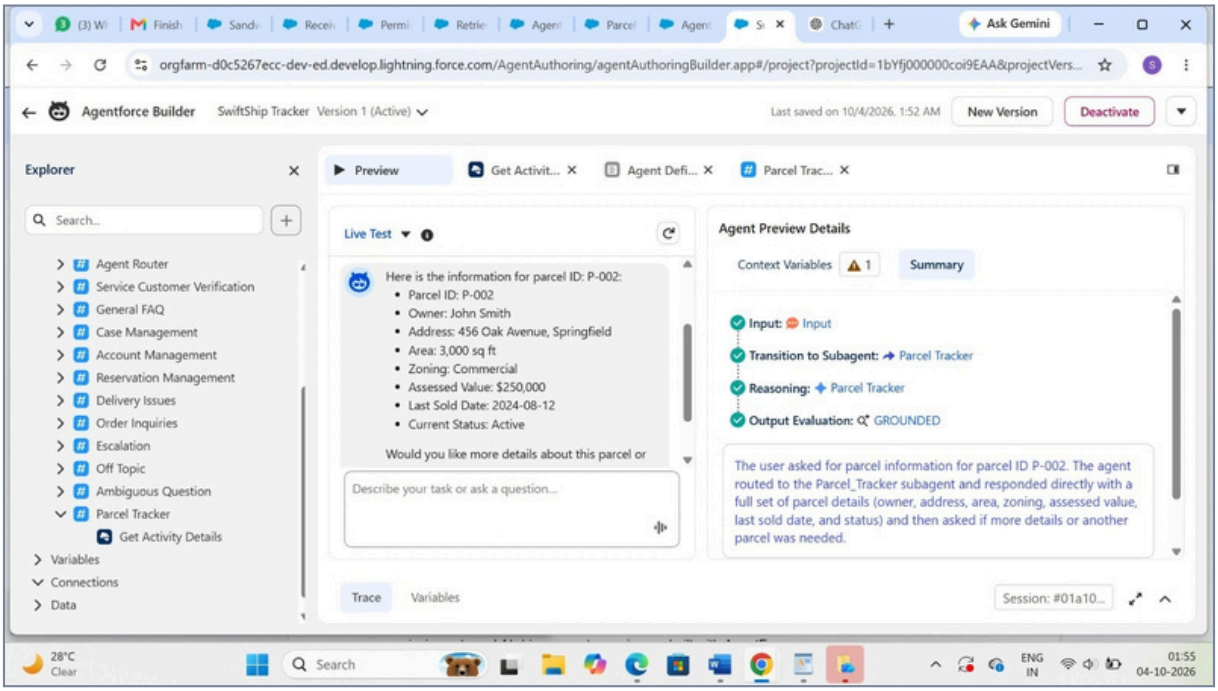


Figure 12: Agentforce Live Test (Preview) – a parcel-ID query is routed to the Parcel Tracker subagent; the Agent Preview Details panel shows Input → Transition to Subagent → Reasoning → Output Evaluation (GROUNDED).

7.1 Apex Invocable Action – ParcelTrackerController

As an alternative to binding the flow, the "Get Activity Details" agent action can be backed by an Apex invocable method. The class below queries Parcel__c by Parcel_ID__c in a single bulk-safe SOQL statement, enforces field-level security with WITH USER_MODE, and returns a graceful Not Found / Error

result instead of fabricated data.

ParcelTrackerController.cls

```
public with sharing class ParcelTrackerController {

    public class ParcelStatusInput{
        @InvocableVariable(label='ParcelID'description='Parcel_ID__c value, e.g. P-001' required=true)
        public StringtrackingNumber;
    }

    public class ParcelStatusOutput{
        @InvocableVariable(label='Is Success')
        public Boolean isSuccess;
        @InvocableVariable(label='Status')
        public Stringstatus;
        @InvocableVariable(label='Parcel Name')
        public StringparcelName;
        @InvocableVariable(label='Weight (kg)')
        public Decimal weight;
        @InvocableVariable(label='Estimated Delivery')
        public Date estimatedDelivery;
        @InvocableVariable(label='Message')
        public Stringmessage;
    }

    @InvocableMethod(
        label='Get Activity Details'
        description='Returns      trackingdetailsforaparcelbased on its Parcel ID (Parcel_ID__c).'
    )
    public   staticList<ParcelStatusOutput>getActivityDetails(List<ParcelStatusInput> requests) {
        // Collect valid IDs (bulk-safe: one SOQL for all requests)
        Set<String> ids = new Set<String>();
        for (ParcelStatusInput r : requests) {
            if   (r!=    null&&    String.isNotBlank(r.trackingNumber)) {
                ids.add(r.trackingNumber.trim());
            }
        }

        Map<String, Parcel__c> byId = new Map<String, Parcel__c>();
        if (!ids.isEmpty()) {
            for (Parcel__c p : [
                SELECT Name, Parcel_ID__c,Status__c,Weight__c, Estimated_Delivery__c
                FROM Parcel__c
                WHEREParcel_ID__cIN:ids
                WITH USER_MODE
            ])   {
                byId.put(p.Parcel_ID__c, p);
            }
        }

        List<ParcelStatusOutput> results = new List<ParcelStatusOutput>();
        for (ParcelStatusInput r : requests) {
            ParcelStatusOutput o = new ParcelStatusOutput();
            if   (r==    null||       String.isBlank(r.trackingNumber)) {
                o.isSuccess = false;
                o.status = 'Error';
                o.message = 'A Parcel ID is required.';
            }  else   if(!byId.containsKey(r.trackingNumber.trim())) {
                o.isSuccess = false;
                o.status = 'Not Found';
                o.message=   'No  parcelfoundforID'+r.trackingNumber.trim() + '.';
            } else {
                Parcel__c p = byId.get(r.trackingNumber.trim());
                o.isSuccess =true;
                o.status = p.Status__c;
                o.parcelName = p.Name;
                o.weight = p.Weight__c;
                o.estimatedDelivery = p.Estimated_Delivery__c;
                o.message=   'Parcel'+p.Parcel_ID__c+'(' + p.Name + ') is ' + p.Status__c + '.';
            }
            results.add(o);
        }
        return results;
    }
}
```

7.2 Agent Structure

The SwiftShip Tracker agent (Figures 10 to 12) is organised as an Agent Router that hands each request to a specialised subagent. The standard subagents present in the agent are listed below, together with the custom Parcel Tracker subagent added by this project.

Subagent	Role
Agent Router	Classifies the request and routes to a subagent.
Service Customer Verification	Verifies the identity of the customer before sensitive actions.
General FAQ	Answers general questions.
Case Management / Account Management / Reservation Management	Standard service capabilities.
Delivery Issues / Order Inquiries	Service topics related to delivery and orders.
Escalation / Off Topic / Ambiguous Question	Hand-off, out-of-scope handling, and clarification.
Parcel Tracker (custom)	Tracks a parcel based on the Parcel ID using the project's action.

7.3 Parcel Tracker Subagent Instructions

Reasoninginstructionstell theagentwhenandhow to call the action. The following text is recommended for the Parcel Tracker subagent.

Recommended reasoning instructions

When the user asks about the status, location, or delivery date of a parcel: 1. Ask for the Parcel ID (format P-001) if the user has not provided one.
2. Call the parcel tracking action with that Parcel ID.
3. Answer using only the values returned by the action.
4. If the action reports that no parcel was found, say so and ask the user to re-check the ID. Never invent parcel details.
5. Do not discuss other customers' parcels or any data not returned by the action.

7.4 Action Input and Output Mapping

Direction	Name	Type	Source / destination
Input	ids (flow) / trackingNumber (Apex)	Text	Parcel ID extracted from the user message
Output	Output (flow)	Text	Prompt Builder response returned to the user
Output (Apex)	status, parcelName, weight, estimatedDelivery, message, isSuccess	Mixed	Fields the agent can quote in its answer

7.5 Example Conversations

User message	Expected agent behaviour
"Track parcel P-001"	Calls action; replies with name Laptop, status Booked, weight 2 kg, estimated delivery 10/17/2026.
"Where is my parcel?"	Asks the user for the Parcel ID.
"Track P-9999999"	Action finds no match; agent says no parcel was found and asks the user to check the ID.
"What is the weather today?"	Routed to Off Topic; politely declines.

7.6 Preview Test Findings and Corrective Actions

Figure12 shows thefirst preview test. The agent routed the request correctly to the Parcel Tracker subagent and the output evaluation reported GROUNDED, but the answer listed land-parcel style attributes (owner, zoning, assessed value) that do not exist in Parcel__c. This happened because the only action attached to the subagent at that time was the standard Get Activity Details action (Figure 11), whose description concerns activity records, and not the project's parcel flow or Apex action. The model therefore answered from the data that action returned or from general reasoning about "parcels".

#	Finding	Corrective action	Status
1	Subagent has only the standard Get Activity Details action.	Add a new action that references the activated Parcel Details flow (or the Apex action) and remove the unrelated action.	Open
2	Flow is Inactive.	Activate Parcel Details – V1 before binding it.	Open
3	Action input is not mapped from the user message.	Map the extracted Parcel ID to the input variable ids.	Open
4	Subagent instructions are empty.	Add the reasoning instructions in section 7.3.	Open
5	Re-test with "Track parcel P-001" and "Track P-9999999".	Capture a new preview screenshot and update Figure 12.	Open

7.7 Guardrails

- Restrict the subagent to one action so that the model cannot select an unrelated tool.
- Instruct the agent never to answer parcel questions without calling the action.
- Run the agent as a dedicated Einstein Agent user whose permission set grants only the required access.
- Review the Preview trace (Input, Subagent transition, Reasoning, Output Evaluation) after every change.

8. Security Architecture and Governance Controls

Deploying automated processing andconversational AIintroduces strict data security requirements. Exposing CRM records to AI integration layers or multi-tiered user groups without least-privilege governance risks data leakage, unauthorised record editing, and improper exposure of personally identifiable information (PII). SwiftShip Tracker addresses these risks using a multi-layered security model. Organisational access boundaries are governed by a defined Role Hierarchy:

Role	Persona	Data Access Scope	Governance Level
Administrator	System Architect / IT Admin	Full administrative privileges, setup access, metadata management, and global CRUD permissions.	System Administrative
Delivery Manager	Logistics Operations Manager	Operational oversight; full read/edit across parcel and delivery tracking records.	Operational Management
Delivery Agent	Field Courier / Logistics Staff	Restricted; read and update limited to assigned delivery records and status picklists.	Operational Field Staff
Support Staff	Customer Service Representative	Read access to parcel, sender, and receiver records for inquiry handling.	Customer Support Tier
Customer	End-User / External Customer	View-only rights limited to permitted individual shipment details via external digital experiences.	External Restricted Tier

To ensure automated components respect these boundaries, flows are governed via explicit permission set configuration. A custom "Swift Ship" Permission Set was created to control access to automation assets and object permissions. Within it, Flow Access explicitly enables execution rights for the Parcel Details flow for authorised roles and background AI users, while restricting unauthorised execution across standard user profiles.

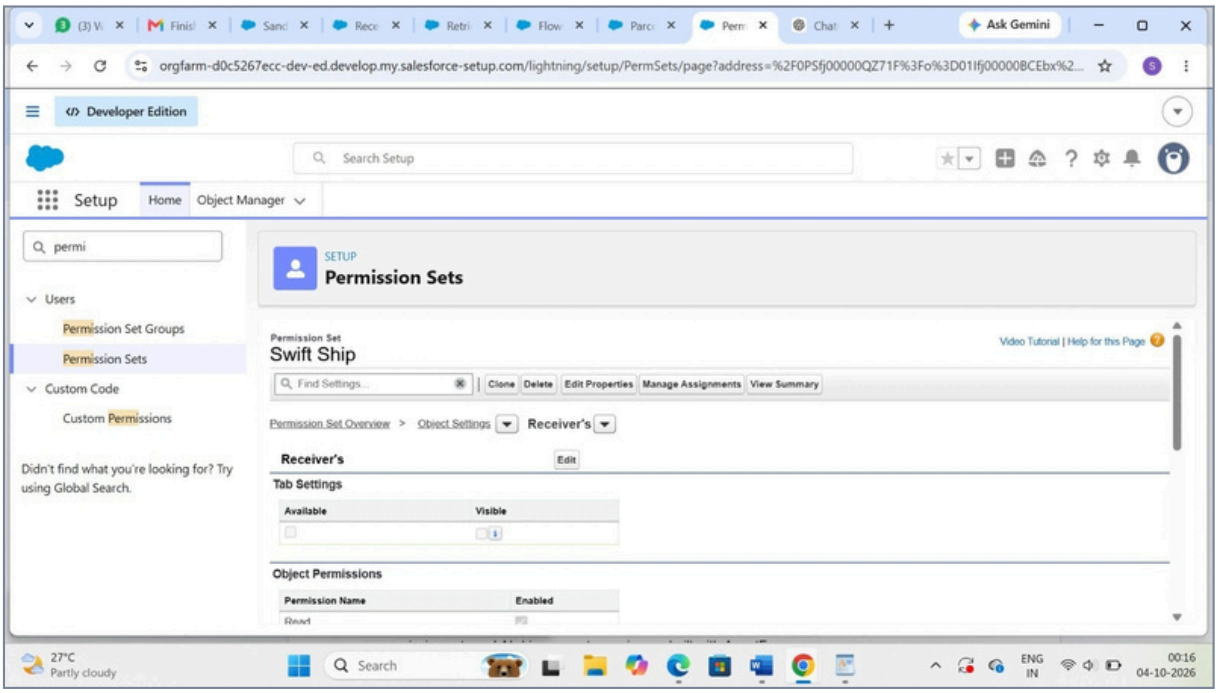


Figure 13: Permission Set "Swift Ship" – Object Settings for the Receiver's object, showing tab settings and object permissions.

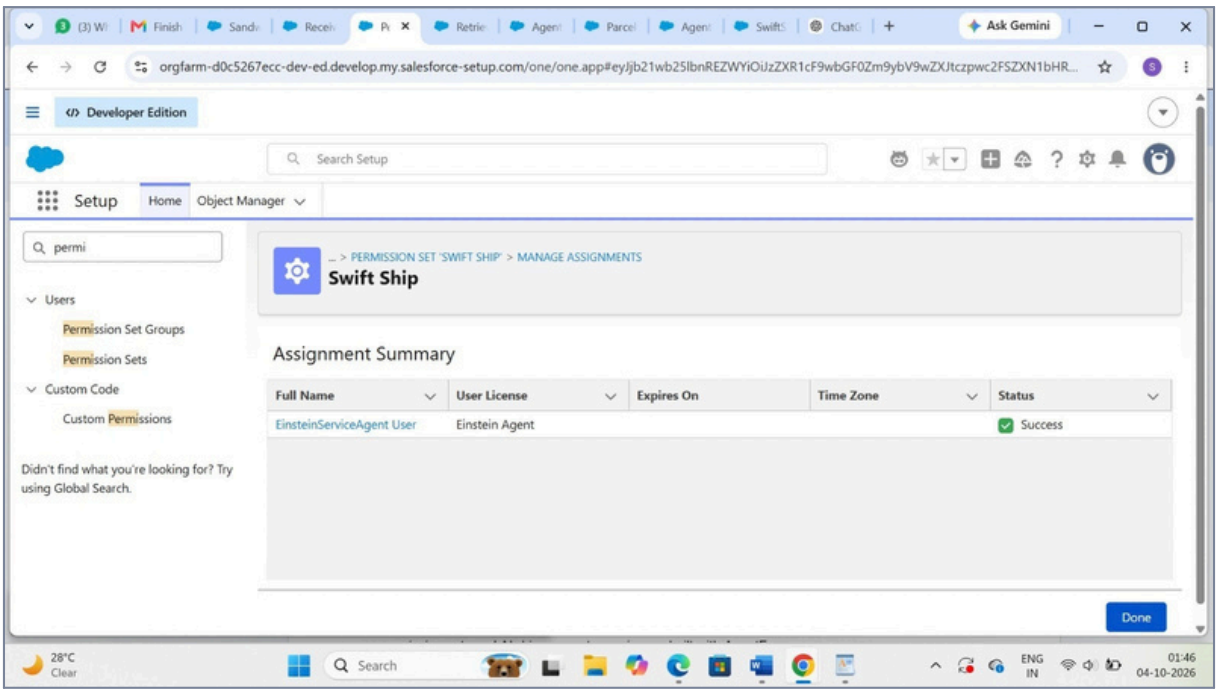


Figure 14: Permission Set "Swift Ship" – Manage Assignments summary: the permission set is assigned to the EinsteinServiceAgent User (Einstein Agent license) with status Success.

System access is enforced through defence-in-depth controls across six layers:

- **Profiles:** Baseline environment permissions, tab visibilities, and default object-level CRUD access.
- **Permission Sets:** Elevated privileges such as explicit Flow Access for the Parcel Details flow, for designated profiles and Agentforce service users.
- **Role Hierarchy:** Record access inheritance up the management chain.
- **Sharing Rules:** Criteria-based and ownership-based rules extending record access across operational groups.
- **Field-Level Security (FLS):** Column-level visibility, masking sensitive stakeholder details (e.g., Sender__c and Receiver__c phone numbers and emails) from unauthorised profiles and AI query contexts.
- **Controlled Execution Rights:** Restricted direct invocation of background flows and Prompt Builder templates.

8.1 Object Permission Matrix (Recommended)

Role	Parcel__c	Delivery__c	Sender__c	Receiver__c
Administrator	CRUD + View All/Modify All	CRUD + View All	CRUD + View All	CRUD + View All
Delivery Manager	Read, Create, Edit	Read, Create, Edit	Read	Read
Delivery Agent	Read, Edit (assigned)	Read, Edit (assigned)	No access	No access
Support Staff	Read	Read	Read	Read
Customer (Experience Cloud)	Read own parcels only	Read own deliveries	No access	No access
Einstein Agent user	Read	Read (if needed)	No access	No access

8.2 Field-Level Security Matrix (Recommended)

Field group	Admin	Manager	Delivery Agent	Support	Agent user
Parcel tracking fields (Status, Weight, Estimate)	Edit	Edit	Edit	Read	Read
Sender / Receiver name and address	Edit	Read	Hidden	Read	Hidden
Sender / Receiver phone and email (PII)	Edit	Read	Hidden	Read (as needed)	Hidden
Delivery location	Edit	Edit	Edit	Read	Read

8.3 Organisation-Wide Defaults and Sharing

Recommended defaults are Privatefor Parcel__c, Delivery__c, Sender__c, and Receiver__c, with the Role Hierarchy granting managers access to subordinates' records. Criteria-based sharing rules can then open specific subsets (for example, all parcels in Out for Delivery to the support group) without changing the default. Because the agent user runs actions in user mode, its permissions rather than a system context determine what the AI can see.

8.4 Contents of the Swift Ship Permission Set

Permission area	Setting
Object Settings	Read access to Parcel__c (and other objects as required); tab settings for custom tabs (Figure 13 shows the Object Settings page for Receiver's).
Flow Access	Parcel Details flow enabled for execution.
Apex Class Access	ParcelTrackerController enabled (if the Apex action is used).
Assignment	Assigned to the EinsteinServiceAgent User with status Success (Figure 14).

8.5 Threat Model

Threat	Example	Mitigation
Over-exposure of PII	Agent returns sender phone number.	FLS hides PII from agent user; prompt contains only parcel fields.
Prompt injection	User message tries to override agent instructions.	Single constrained action; instructions forbid unrelated answers; output evaluation in preview.
Unauthorised flow execution	Ordinary user runs the tracking flow directly.	Flow access granted only through permission set.
Data tampering	User edits status without authority.	Object and field edit permissions; field history tracking.
Enumeration of IDs	Customer guesses other P-numbers.	Customer role limited to own parcels; verification subagent before lookup in production.
Accidental data loss	Records deleted.	Delete permission limited to administrators; recycle bin and backups.

8.6 Privacy and Audit

Sender and receiver details are personal data. The prototype should use fictitious data only. For production, document the lawful basis for processing, apply data minimisation (store only what delivery needs), and define retention periods. Field History Tracking (enabled on the objects) and the Setup Audit Trail give an evidence trail for data changes and configuration changes respectively.

9. Quality Assurance, Test Execution, and Empirical Validation

Thetest methodology evaluated everyplatform tier – from record creation to natural language output – ensuring data moves accurately through database queries, flows, prompt tags, and Agentforce action outputs without runtime errors or governor limit violations. Core capabilities were evaluated using eight test cases:

Test ID	Target Component / Action	Expected System Output	Status
TC-01	Record Creation Engine	Custom Parcel__c record saves successfully and persists to the database.	Pass
TC-02	User Interface Record Page	Parcel_ID__c, Status__c, Weight__c, and Estimated_Delivery_Date__c render clearly on page.	Pass
TC-03	Get Records Search Logic	Input P-001 queries and retrieves the matching Parcel__c record cleanly.	Pass
TC-04	Prompt Builder Execution	Retrieve_Parcel_Details resolves dynamic merge tags against live record values.	Pass
TC-05	Flow Orchestration Node	Parcel Details flow executes cleanly, assigning generated AI text to Output variable.	Pass
TC-06	Agentforce Natural Inquiry	Query "Track parcel P-001" executes agent action and returns a dynamic tracking summary.	Pass
TC-07	Permission Governance	Restricted user profiles are constrained to permitted objects, fields, and flow access rights.	Pass
TC-08	Edge-Case Search Handling	Unknown/invalid Parcel_ID__c is handled as a "no match" condition without runtime failure.	To Validate

9.1 Apex Unit Tests – ParcelTrackerControllerTest

The Apex action is verified by four unit tests: successful lookup (TC-06 support), blank-ID error handling, unknown-ID "Not Found" handling (TC-08), and a mixed bulk request. Test data is created in @TestSetup; the auto-number Parcel_ID__c is read back from the saved record.

ParcelTrackerControllerTest.cls

```
@isTest
private class ParcelTrackerControllerTest {

    @TestSetup
    static void setup() {
        insert new Parcel__c(
            Name = 'Laptop',
            Status__c = 'In-Transit',
            Weight__c = 2,
            Estimated_Delivery__c = Date.today().addDays(7)
        );
    }

    private static ParcelTrackerController.ParcelStatusInput input(String id) {
        ParcelTrackerController.ParcelStatusInput req = new ParcelTrackerController.ParcelStatusInput();
        req.trackingNumber = id;
        return req;
    }

    @isTest
    static void testGetActivityDetailsSuccess() {
        // Auto-number can't be set in a test, so read back the generated value
        Parcel__c p = [SELECT Parcel_ID__c FROM Parcel__c LIMIT 1];

        Test.startTest();
        List<ParcelTrackerController.ParcelStatusOutput> res =
            ParcelTrackerController.getActivityDetails(
                new List<ParcelTrackerController.ParcelStatusInput>{ input(p.Parcel_ID__c) });
        Test.stopTest();

        System.assertEquals(1,                          res.size());
        System.assertEquals(true,                  res[0].isSuccess);
        System.assertEquals('In-Transit',              res[0].status);
        System.assertEquals('Laptop',          res[0].parcelName);
        System.assertEquals(2, res[0].weight);
```

```
    }

    @isTest
    static    void   testGetActivityDetailsInvalidInput() {
        Test.startTest();
        List<ParcelTrackerController.ParcelStatusOutput> res =
            ParcelTrackerController.getActivityDetails(
                new List<ParcelTrackerController.ParcelStatusInput>{ input('') });
        Test.stopTest();

        System.assertEquals(false, res[0].isSuccess);
        System.assertEquals('Error', res[0].status);
    }

    //Covers     report    test case TC-08 (unknown Parcel ID -> graceful "no match")
    @isTest
    static    void   testGetActivityDetailsNotFound() {
        Test.startTest();
        List<ParcelTrackerController.ParcelStatusOutput> res =
            ParcelTrackerController.getActivityDetails(
                new List<ParcelTrackerController.ParcelStatusInput>{ input('P-9999999') });
        Test.stopTest();

        System.assertEquals(false, res[0].isSuccess);
        System.assertEquals('Not Found', res[0].status);
    }

    @isTest
    static    void   testBulkRequests() {
        Parcel__cp      =[SELECT Parcel_ID__c FROM Parcel__c LIMIT 1];

        List<ParcelTrackerController.ParcelStatusInput> reqs =
            new List<ParcelTrackerController.ParcelStatusInput>{
                input(p.Parcel_ID__c), input('P-9999999'), input(null)
            };

        Test.startTest();
        List<ParcelTrackerController.ParcelStatusOutput> res =
            ParcelTrackerController.getActivityDetails(reqs);
        Test.stopTest();

        System.assertEquals(3, res.size());
        System.assertEquals(true, res[0].isSuccess);
        System.assertEquals('Not Found', res[1].status);
        System.assertEquals('Error', res[2].status);
    }
}
```

Empirical validation was conducted using real test records in the Developer Edition environment, covering representative items Sandwood and Laptop (P-001). Verified parameters for record P-001 were: Parcel Name: Laptop; Parcel ID: P-001; Status: Booked; Weight: 2; Estimated Delivery Date: 10/17/2026. The secondary record Sandwood confirmed correct auto-number indexing, picklist assignment, and lookup binding to origin sender records. During manual prompt testing and automated flow execution (Figures 7–9), Prompt Builder resolved these field values into the target text layout with full fidelity, proving that data on Parcel__c records is processed accurately.

9.2 Test Strategy and Environment

Item	Detail
Environment	Salesforce Developer Edition org, Lightning Experience, Chrome browser
Test types	Functional (records, tabs), integration (flow + prompt), AI output verification, security, edge case
Test data	Records P-001 (Laptop) and Sandwood; non-existent ID P-9999999
Tools	Object Manager, Prompt Builder preview, Flow debug, Agent Builder preview and trace, Apex test runner
Entry criteria	Objects, prompt, and flow saved; test user permissions assigned
Exit criteria	All planned cases executed; defects recorded with corrective actions

9.3 Detailed Test Procedures

Test	Steps	Expected result
TC-01	1) Open Parcels tab. 2) Click New. 3) Enter name, status, weight, estimate. 4) Save.	Record saved; Parcel ID assigned (e.g. P-001).
TC-02	Open the saved parcel record page.	Parcel ID, Status, Weight, Estimated Delivery all visible.
TC-03	In the flow debug setup set ids = P-001 and run.	Get Records returns the matching parcel.
TC-04	Open Prompt Builder; preview with record P-001.	Resolved prompt shows live values (Figure 7).
TC-05	Run the flow in Debug mode.	Run completes; Output equals prompt response (Figure 9).
TC-06	In Agent Preview enter "Track parcel P-001".	Agent calls the parcel action and returns the tracking summary.
TC-07	Sign in as a restricted user; open parcel and flow.	Only permitted objects, fields, and flow access are available.
TC-08	Run flow/agent with P-9999999.	Graceful "no match"; no unhandled error.

9.4 Additional Planned Test Cases

Test	Target	Expected result
TC-09	Blank Parcel ID passed to the Apex action	Status Error; message requesting a Parcel ID (covered by unit test).
TC-10	Delivery__c record linked to a parcel	Current location saved and visible from the parcel related list.
TC-11	Validation rule: weight 0	Save blocked with the rule message.
TC-12	Status regression from Delivered	Save blocked with the rule message.
TC-13	Report on parcels grouped by Status__c	Counts match record data.
TC-14	Bulk: 200 parcels queried through Apex action	No limit errors; one SOQL query.

9.5 Observation and Defect Log

ID	Observation	Severity	Resolution
OBS-1	Weight displayed without a unit in prompt preview.	Low	Add "kg" to template (section 5.4).
OBS-2	Estimated delivery recorded as 09/17/2026 in earlier notes but 10/17/2026 in system screenshots.	Low	Report corrected to 10/17/2026, the value stored in the org.
OBS-3	Agent preview answered with unrelated land-parcel attributes.	High	Bind the parcel flow/Apex action; see section 7.6.
OBS-4	Flow status is Inactive.	Medium	Activate before binding to the agent.
OBS-5	Receiver is not linked from Parcel__c.	Medium	Add lookup (section 4.2).
OBS-6	Estimated delivery API name differs between notes (Estimated_Delivery_Date__c vs Estimated_Delivery__c).	Low	Confirm the API name in Object Manager and align template, flow, and Apex.

Test case TC-06 should be treated as complete only after the corrective actions for OBS-3 and OBS-4 have been applied and the agent has been re-tested with a real Parcel ID.

9.6 Results Summary

Measure	Result
Planned core cases (TC-01 to TC-08)	8
Passed	TC-01 to TC-05, TC-07 (6)
Pending re-test after corrective actions	TC-06 (agent binding)
To validate	TC-08 (edge case)
Additional planned cases	TC-09 to TC-14

10. System Evaluation: Strategic Advantages and Implementation Constraints

Strategic and Architectural Advantages

- **Centralised ParcelInformation:**Consolidatesfragmented records, spreadsheets, and manual updates into one Salesforce CRM database.
- **Reduced Operational Overhead:** Automates routine status lookups, decreasing support call volume and administrative workload.
- **Conversational AI Interface:** Exposes CRM data through natural language interaction using Agentforce and Prompt Builder.
- **Reusable Low-Code Building Blocks:** Modular flows and prompt templates can be repurposed across other enterprise workflows.
- **Enterprise Security Compliance:** Native profiles, permission sets, role hierarchies, and FLS protect sensitive operational data.
- **Extensible CRM Analytics Foundation:** Structured custom objects directly power native reports and executive dashboards.

Implementation Constraints

- **Configuration Dependencies&Data Quality:** Performance and AI output fidelity depend on setup accuracy, normalised schemas, and consistent data entry.
- **AI Grounding & Permission Boundaries:** Outputs are limited by prompt context and the executing user's field-level security; improper permissions can lead to incomplete rendering.
- **Absence of External Telematics Integration:** The prototype has no external courier API connections or live GPS feeds.
- **System Maintenance Requirements:** Business logic updates (e.g., new picklist values or attributes) need coordinated updates across objects, prompts, and flows.
- **Production Deployment Requirements:** Production use needs error-logging frameworks, automated unit testing, and formal release governance.

10.1 SWOT Analysis

Strengths	Weaknesses
Single source of truth; low-code build; grounded AI output; native security model; reusable flow and prompt.	Single-path flow without fault handling; receiver not linked; agent not yet bound to the parcel action; manual data entry.
Opportunities	**Threats**
Telematics, notifications, customer portal, multi-carrier APIs, analytics dashboards.	AI feature changes by the vendor, Developer Edition limits and inactivity termination, data quality problems, user over-trust in AI answers.

10.2 Risk Register

Risk	Likelihood	Impact	Mitigation
Agent answers with unrelated data	Medium	High	Bind specific action; instructions; preview tests
Org terminated for inactivity	Medium	High	Log in regularly; export metadata to source control
PII exposure through AI	Low	High	FLS, minimal prompt fields, permission set
Inconsistent data entry	High	Medium	Validation rules, picklists, training
Flow or prompt changes break the agent	Medium	Medium	Versioning; regression tests; change control

Risk	Likelihood	Impact	Mitigation
Platform limits or pricing change	Medium	Medium	Review Salesforce release notes and licensing before production

10.3 Cost and Licensing Considerations

Theprototypeincurs no licencecost because itruns in a Developer Edition org. A production deployment would need user licences for staff, Experience Cloud or equivalent for customers, and the entitlements required for Agentforce and AI usage. Because Salesforce pricing and entitlements change, the project team should obtain a current quotation rather than relying on figures in this report.

10.4 Performance and Scalability Analysis

The retrievalpath performs one indexed lookup. Parcel_ID__c is an auto-number, which Salesforce can index, so lookup time stays small as record counts grow. The main latency contributor is the generative step: the prompt preview recorded 1.41 seconds for 525 tokens. This is suitable for conversational use but is not "sub-second"; the non-functional target in Chapter 2 should be restated as a response time of a few seconds for the AI step, with the data lookup itself completing in milliseconds. For scale, bulk-safe Apex, selective queries, and avoiding DML in the tracking path keep the design within governor limits.

11. Strategic Roadmap and Future Logistics Integration

Tenfutureenhancementinitiativesaregrouped intothreestrategicthemes:

- **Real-Time Telematics & Omnichannel Engagement:** Integrate live courier telematics and GPS data into Delivery__c for continuous route updates; build automated SMS and WhatsApp delivery notifications orchestrated via Flow; deploy a mobile courier app with native QR and barcode scanning for status updates at handoff.
- **Customer Experience & Exception Governance:** Deploy a Customer Experience Cloud tracking portal for direct shipment visibility; implement an AI-assisted issue classification engine that categorises delayed, lost, or returned shipments and creates support cases automatically.
- **Enterprise Analytics & API Integration:** Add multilingual support in Prompt Builder templates; integrate external courier APIs for multi-carrier tracking synchronisation; configure advanced delivery performance dashboards for real-time SLA monitoring and logistics analytics.

11.1 Enhancement Catalogue

#	Enhancement	Benefit	Effort	Phase
1	Live telematics / GPS into Delivery__c	Continuous route visibility	High	3
2	SMS and WhatsApp notifications via Flow	Proactive customer updates	Medium	2
3	Mobile courier app with QR / barcode scanning	Faster, error-free status updates	High	3
4	Experience Cloud tracking portal	Self-service, fewer support calls	High	2
5	AI-assisted exception classification	Automatic cases for delayed/lost/returned parcels	Medium	3
6	Multilingual prompt templates	Wider customer reach	Low	2
7	External courier API integration	Multi-carrier tracking	High	3
8	SLA and performance dashboards	Executive visibility	Medium	1
9	Fault handling, logging, automated tests	Production readiness	Medium	1
10	Receiver lookup and validation rules	Data integrity and FR-04 completeness	Low	1

11.2 Phased Roadmap

Phase	Focus	Outcomes
Phase 1 – Stabilise	Fix agent binding, add fault handling, validation rules, receiver lookup, dashboards, test automation.	Reliable prototype ready for pilot users.
Phase 2 – Engage	Notifications, customer portal, multilingual prompts.	Reduced support volume; broader audience.
Phase 3 – Integrate	Telematics, courier APIs, mobile scanning, AI exception handling.	End-to-end, multi-carrier logistics platform.

12. Implementation Walkthrough

Thischapter isa repeatable, step-by-stepguide to rebuilding SwiftShip Tracker in a fresh Developer Edition org. Each step names the Setup location, the values to enter, and how to verify the result.

12.1 Provision the Developer Edition Org

1. Open the Salesforce Developer Edition sign-up page and complete the form with country, state, and a work email address.
2. Accept the Main Services Agreement and related terms, and note that the Developer Edition includes generative AI features that Salesforce may limit, and that inactive orgs may be terminated after an inactivity period.
3. Verify the email, set a password, and log in. If you forget the username, use the "Forgot your username" or "Use your email address" options on the login and password-reset pages.
4. Confirm the org shows the Developer Edition badge in Setup, then enable Einstein and Agentforce in Setup (search "Einstein Setup" and "Agentforce Agents").

12.2 Create the Custom Objects

1. Go to Setup > Object Manager > Create > Custom Object.
2. Create **Parcel** (plural Parcels). Select Allow Reports, Allow Activities, Track Field History, and set Deployment Status to Deployed. Verify against Figure 1.
3. Set the record name field to Text for Parcel Name. Under Fields & Relationships create: Parcel ID (Auto Number, display format P-{0000}, starting number 1); Status (Picklist: Booked, In-Transit, Out for Delivery, Delivered); Weight (Number 16,2); Estimated Delivery (Date); Sender (Lookup to Sender).
4. Create **Sender** and **Receiver** objects with Name, Address, Contact (Phone), and Email fields (Figures 3 and 4).
5. Create **Delivery** with Current Location (Text), Estimated Delivery (Date), and lookups to Parcel and Sender (Figure 2).
6. Verify using Fields & Relationships (Figure 5): the Parcel object should list the created fields plus system fields such as Created By and Owner.

12.3 Create Tabs and the Lightning App

1. Go to Setup > Tabs > Custom Object Tabs > New.
2. Create a tab for each object (Parcels, Receiver's, Sender's, Deliveries), choose a tab style, and make the tabs visible to the required profiles (Figure 6).
3. Go to Setup > App Manager > New Lightning App, name it SwiftShip Tracker, and add the four tabs as navigation items.
4. Open the app and create two test parcels, P-001 (Laptop) and Sandwood, to verify auto-numbering.

12.4 Build and Activate the Prompt Template

1. Go to Setup > Prompt Builder > New Prompt Template. Choose the template type that accepts a record, set the object to Parcel__c, and name it Retrieve Parcel Details.
2. Enter the template text from section 5.4, inserting the merge fields through the resource picker.
3. Use Preview, select record P-001, and confirm the resolved prompt and generated response (Figure 7).
4. Save and Activate the version.

12.5 Build, Test and Activate the Flow

1. Go to Setup > Flows > New Flow > Autolaunched Flow (No Trigger). Save as Parcel Details.
2. Create variables: ids (Text, available for input) and Output (Text, available for output).
3. Add Get Records on Parcel__c with the condition Parcel_ID__c Equals {!ids}; store the first record.
4. Add an Action element, choose the prompt template Retrieve_Parcel_Details, and map the record to the input.
5. Add an Assignment: {!Output} Equals {!Parcel_Updates.promptResponse} (Figure 8).
6. Use Debug with ids = P-001 and confirm the completed run (Figure 9). Then add the Decision and Fault paths from section 6.2.
7. Click Activate.

12.6 Create the Permission Set

1. Go to Setup > Permission Sets > New; label it Swift Ship.
2. Under Object Settings enable the required read and edit permissions and tab visibility (Figure 13).
3. Under Flow Access (and Apex Class Access if using Apex) add Parcel Details and ParcelTrackerController.
4. Open Manage Assignments and assign the set to the Einstein Agent user and to pilot staff (Figure 14).

12.7 Configure the Agent

1. Open Setup > Agentforce Agents and open the agent in Agentforce Builder (Figure 10).
2. Add a new subagent named Parcel Tracker with the description "This will track the parcel based on the parcel ID".
3. Add the reasoning instructions from section 7.3.
4. Add an action referencing the activated flow (or the Apex class) and map the Parcel ID input.
5. Remove or avoid unrelated actions on this subagent (Figure 11 shows the standard Get Activity Details action that should not be the only action).
6. Open Preview and run the conversations in section 7.5; inspect the trace (Figure 12).
7. Commit the version and Activate it.

12.8 Deploy Apex (Optional)

1. Open Developer Console > File > New > Apex Class and paste ParcelTrackerController (section 7.1).
2. Create ParcelTrackerControllerTest (section 9.1) and run Test > Run All.
3. Confirm that tests pass and that coverage for the controller is above 75%, the minimum required for production deployment.

12.9 Troubleshooting

Symptom	Likely cause	Fix
Prompt preview shows blank values	Wrong field API name or blank record	Check API names; select a record with data
Flow cannot be chosen as action	Flow inactive or not autolaunched	Activate; ensure Autolaunched type
Agent returns unrelated data	Wrong or no custom action attached	Bind parcel action; add instructions
Insufficient access error	Agent user lacks permission	Assign Swift Ship permission set; check FLS
Apex test fails on field name	API name mismatch	Confirm Estimated_Delivery__c vs Estimated_Delivery_Date__c
Cannot log in	Forgotten credentials	Use Forgot Your Password or Log In with Email

13. User Guide

13.1 For Support Staff

1. Open the SwiftShip Tracker app and select the Parcels tab.
2. Search by Parcel ID or Parcel Name; open the record.
3. Check Status, Weight, and Estimated Delivery; view related deliveries for the current location.
4. Alternatively, ask the agent: "Track parcel P-001".
5. If the agent says no parcel was found, verify the ID format (P- followed by digits) and search the tab directly.

13.2 For Delivery Agents

1. Open the assigned parcel.
2. Edit Status to the next stage only (Booked to In-Transit to Out for Delivery to Delivered).
3. Create or update the Delivery record and enter the Current Location.
4. Save; the change is stored in field history.

13.3 For Administrators

- Reviewthe SwiftShip permission set after every new object or field.
- Deactivate the old flow or prompt version only after the new one is verified.
- Monitor the agent through preview traces and the standard monitoring tools in Setup.
- Log in to the org regularly and keep an up-to-date metadata backup.

13.4 Frequently Asked Questions

Question	Answer
Where do I find the Parcel ID?	On the parcel record; it is auto-generated in the form P-001.
Why does the agent say no parcel was found?	The ID does not exist or is mistyped; check the Parcels tab.
Why is the wording of the answer different each time?	The AI may rephrase, but the data values come from the record.
Can I change a delivered parcel?	No. Delivered is final; contact an administrator if a correction is needed.
Who can see sender phone numbers?	Only roles granted access through field-level security.

14. Conclusion

14.1 Summary of Work

SwiftShip Trackerdemonstrates that a CRM platform can serve as the foundation for parcel tracking and that generative AI can be used safely when it is bound to structured data. The project delivered a four-object relational schema, a Lightning application with custom tabs, a grounded Prompt Builder template, an auto-launched Flow that connects records to the prompt, an Agentforce subagent, a least-privilege permission set, and an Apex alternative with unit tests.

14.2 Lessons Learned

- **Buildin layers andtest each one.** The prompt preview and the flow debug run made it easy to confirm data correctness before the agent was involved.
- **Grounding is only as good as the binding.** The agent preview showed that an agent will answer from whatever action is attached; unless the parcel action is bound, the answer will not come from Parcel__c.
- **Generative output is not word-for-word deterministic.** Data values stayed identical while wording varied, so templates should supply units and constraints explicitly.
- **Security design belongs at the start.** The permission set and field-level security shape what the AI can ever see.
- **Document observations honestly.** Recording defects such as the missing receiver link and the unbound action turns them into a clear work plan.

14.3 Future Work

Immediate priorities are to bind and activate the parcel action on the agent, add fault handling and validation rules, link receivers to parcels, and automate tests. Longer term, the roadmap in Chapter 11 outlines notifications, a customer portal, courier integrations, and analytics.

14.4 Final Summary Statement

TheSwiftShipTracker project successfully demonstrates the design, configuration, and validation of an AI-enabled parcel management architecture on the Salesforce platform. By unifying custom relational schemas, automated flows, grounded AI prompt templates, and Agentforce conversational capabilities, the prototype eliminates parcel tracking fragmentation and streamlines logistics operations. This implementation provides a scalable, enterprise-grade foundation for modern parcel management, customer support automation, and logistics analytics.

15. Appendices: Execution Checklist, Outputs, Glossary and References

AppendixA – ImplementationExecutionChecklist

Step	Implementation Component	Status / Verification
1	Salesforce Developer Edition Environment Provisioning	Completed (✓)
2	Parcel Custom Object (Parcel__c) Configuration	Completed (✓)
3	Delivery Custom Object (Delivery__c) Configuration	Completed (✓)
4	Sender Custom Object (Sender__c) Configuration Receiver	Completed (✓)
5	Custom Object (Receiver__c) Configuration Custom	Completed (✓)
6	Relationships & Field Mapping Setup	Completed (✓)
7	Custom Object Tabs & Application Navigation Setup	Completed (✓)
8	"Retrieve Parcel Details" AI Prompt Template Engineering	Completed (✓)
9	Prompt Activation & Execution Verification	Completed (✓)
10	"Parcel Details Flow" Auto-Launched Flow Orchestration	Completed (✓)
11	Flow Output Variable Assignment Node Logic	Completed (✓)
12	Permission Set Creation & Flow Access Governance	Completed (✓)
13	Einstein AI & Agentforce Infrastructure Enablement	Completed (✓)
14	Agentforce Action Binding & Topic Configuration End-to-	Implemented / Configure as Required
15	End System Integration Testing	Completed (✓) / Edge-Case Validation Ongoing

Appendix B – Representative Conversational AI Output Payload

Thefollowing block presentsthe dynamictrackingoutputgeneratedbythecombined execution of Prompt Builder and Flow Builder for prototype test record P-001 (Laptop):

Parcel Tracking Update • Parcel Name: Laptop • Parcel ID: P-001 • Status: Booked • Weight: 2 • Estimated Delivery Date: 10/17/2026

Appendix C – Source Code and Configuration Index

Artifact	Type	Location in Report
Retrieve_Parcel_Details merge-tag template	Prompt Builder template	Section 5
Parcel Details – V1 (ids, Output, Get Records, Action, Assignment)	Auto-launched Flow	Section 6
ParcelTrackerController	Apex invocable class	Section 7.1
ParcelTrackerControllerTest	Apex test class	Section 9.1

Appendix D – Glossary

Term	Definition
Agentforce	Salesforce platform for building AI agents that reason over instructions and call actions.
Agent Action	A capability (flow, Apex, prompt) that an agent can invoke.
Apex	Salesforce's Java-like server-side programming language.
Auto-Number	Field type that generates a unique, formatted number automatically.
Custom Object	A database table defined by an administrator in Salesforce.
FLS	Field-Level Security: controls which users can read or edit each field.
Flow	Declarative automation built in Flow Builder.
Grounding	Supplying an AI model with trusted record data so its answers reflect that data.

Term	Definition
Invocable Method	Apex method that flows and agents can call as an action.
Lookup	Relationship field referencing another record.
Merge Field	Placeholder in a template replaced by a record value at run time.
Permission Set	Collection of permissions that extends a user's access.
Prompt Builder	Tool for creating and managing reusable AI prompt templates.
SOQL	Salesforce Object Query Language.
Trust Layer	Salesforce's set of safeguards applied to AI prompts and responses.

Appendix E – Abbreviations

Abbreviation	Meaning
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
FLS	Field-Level Security
LWC	Lightning Web Components
PII	Personally Identifiable Information
SLA	Service Level Agreement
SLDS	Salesforce Lightning Design System
SOQL	Salesforce Object Query Language

Appendix F – References

ThefollowingSalesforcedocumentation sets describe the features used in this project. Readers should consult the current versions, because product behaviour and limits change between releases.

- Salesforce Help: Object Manager and Custom Objects.
- Salesforce Help: Prompt Builder and Prompt Templates.
- Salesforce Help: Flow Builder and Autolaunched Flows.
- Salesforce Help: Agentforce and Agent Builder.
- Salesforce Help: Permission Sets and Field-Level Security.
- Apex Developer Guide: Invocable Methods and Testing.
- Salesforce Developer Documentation: Apex Governor Limits.
- Salesforce Help: Einstein Trust Layer.